AUTONOMOUS DELIVERY VEHICLE

BY

Laith Fakhruldin and Spencer Martinez

ET 4999 Senior Project

Submitted to the Faculty of

Division of Engineering Technology

Wayne State University

Detroit, Michigan

in partial fulfillment of the requirements

for the degree of

BACHELOR OF SCIENCE IN COMPUTER TECHNOLOGY

December, 2023

AUTONOMOUS DELIVERY VEHICLE

BY

Laith Fakhruldin and Spencer Martinez

ET 4999 Senior Project

Submitted to the Faculty of

Division of Engineering Technology

Wayne State University

Detroit, Michigan

in partial fulfillment of the requirements

for the degree of

BACHELOR OF SCIENCE IN COMPUTER TECHNOLOGY

December, 2023

Approved by:

Professor Moise Sunda

09/06/2023

COPYRIGHT BY
LAITH FAKHRULDIN AND SPENCER MARTINEZ
2023
ALL RIGHTS RESERVED

SUMMARY

AUTONOMOUS DELIVERY VEHICLE

BY

LAITH FAKHRULDIN AND SPENCER MARTINEZ

December, 2023

Advisor: Professor Moise Sunda

Curriculum: Engineering Technology

Degree: Bachelor of Science in Computer Technology

Today, many of our current jobs can be automated without the use of human interaction and can be more efficient from a time and cost perspective. As technology continues to develop and improve, we can greatly eliminate the margin of error in certain industries and reduce the time required to perform tasks, especially for package delivery drivers, because newer employees may not be familiar with the landscape or lack the motivation to complete their job in a timely manner.

Our team is creating an autonomous Arduino vehicle that takes advantage of high-quality infrared, ultrasonic, and RGB color detecting sensors, allowing the vehicle to obtain information about its surroundings and determine which actions need to be performed. This would be accomplished by using the C++ programming language as back-end and Python as front-end user interface, and we plan to use the Arduino IDE software to incorporate the necessary libraries for this project with Visual Studio as coding platform for Python section. By using sensor technology to operate these repetitive tasks, especially in a warehouse environment, companies can now redirect their focus on teams that require complex decision making and allows the machines to complete the simple, important tasks.

When evaluating our vehicle's performance, we have ensured that it met our functional requirements (described later in this document) along with our engineering standards that we

abide by. Specifically, our vehicle complies with the ISO 9001 standard which ensures that customer expectations are met, allows organizations to continuously improve their processes/systems, and enhances their overall performance (*ISO 9001:2015 - What Is the 9001:2015 Standard? | ASQ*, n.d.). When it comes to customer expectations, our team is ensuring that objects are successfully being moved from one location to another while ensuring that package contents are intact. Our project involves numerous stages such as designing, thorough testing, maintenance, and more. By carefully revisiting each of these areas, even if the current stage varies greatly from the previous one, we can continuously improve our product by taking the required actions based on our new findings.

Table of Contents

List of Tables 8

List of Figures 9

Introduction 10

Literature Review 11

Autonomous Vehicle Functional Requirements 12

Autonomous Vehicle Design 13

Hardware for Autonomous Vehicle 14

LAFVIN Smart Car Robot 15

LAFVIN Pin Allocations 16

CD74HC4067 and TCA9548A Multiplexers 18

PCA9685 Servo Motor Driver Board 19

KY-034 7-Color LED Sensor 20

HR-SR04 Ultrasonic Sensor 21

TCRT5000 Infrared Signal Reflective Tracking Sensor 22

Choosing the Right Sensor for Picking and Dropping Stations 24

TCS34725 RGB Color Recognition Sensor 25

HC-05 Bluetooth communications 27

3D Printing with TinkerCAD 29

Hardware Recommendations 30

Vehicle Hardware Summary 32

Software Development and Implementation 32

Front-End Python Application 33

Code Explanation 34

Python Code 35

Python Application Recommendations 37

Arduino Code 38

Arduino Code Recommendations 43

Socioeconomic and Environmental Effects 44

Test Case Scenarios: ***45***

Final Results ***47***

Lessons Learned ***48***

Lesson 1 48

Lesson 2 49

Conclusion ***50***

References ***52***

Appendices ***54***

Python Code 54

Arduino Code 64

Logbook 116

Progress Updates 122

List of Tables

Table 1 shows the Gantt chart of our Autonomous Delivery Vehicle 122

List of Figures

Figure 1: LAFVIN smart car robot bought from Amazon.com 15
Figure 2: CD7HC4067 analog multiplexer to allocate more pins 18
Figure 3: TCA9548A multiplexer to add more pins to microcontroller 18
Figure 4: PCA9685 servo motor driver board helps reallocate pins in microcontroller 20
Figure 5: Using KY-034 LED light sensor as alarm system when obstacles occur 20
Figure 6: HC-SR04 ultrasonic sensor is used to obstacle avoidance 21
Figure 7: TCRT5000 IR sensors are used as tracking mechanism 22
Figure 8: TCS34725 RGB color detection sensor, played the main role in our project 25
Figure 9: HC-05 is the central communication module that connected the both programs 27
Figure 10: Using TinkerCAD are the main design for the 3D printed parts 29
Figure 11: Claw are made by the 3D printer to accommodate a wider angle for usability 30
Figure 12: Python application used as front-end platform to control the driverless vehicle 33
Figure 13: Using a pop-up window to insert the serial communication port 34

Introduction

Our team has discussed the evolution of our technology and observed that there is more potential in the hardware world than ever before when it comes to implementing technology into the workplace. Specifically, we believe that there are opportunities to take advantage of a variety of sensors in the industry that can handle mundane, repetitive tasks that are being performed by humans. While our team is not aiming to revoke jobs from hard-working individuals, we believe that our technology and its capabilities can be expanded to a broader audience, allowing knowledgeable professionals in different industries to form their own useful piece of equipment, like what our team managed to accomplish.

When it comes to understanding the problem that our project aims to resolve, we are aiming to draw attention to the industry by showing how impressive and unique some products can be and the effect that they can have on our society. Additionally, we want to express the outcomes that are possible when having a piece of technology like ours in a warehouse environment. When asking most users about their thoughts on autonomous technology, most would say that it is not ready to become implemented at the organizational level. However, when deciding on a project that our group was going to create, we believed that this could become a reality, depending on the specific set of operations that are needing to be performed and how critical they are.

Autonomous vehicles that are driving on the road have numerous challenges that need to be faced such as weather conditions, cyber security threats, traffic conditions, and many more. Because our vehicle would be functioning in an indoor environment, does not require the need to connect to a network, or need to be concerned with traffic conditions (specifically other vehicles on the same path), our vehicle should have a substantial advantage when it comes to

affordability, accuracy, and security. Our team is creating an autonomous Arduino vehicle that takes advantage of high-quality infrared, ultrasonic, and light sensors, allowing the vehicle to obtain information about its surroundings and determine which actions need to be performed. This vehicle will prioritize employee safety by monitoring the activity that is performed in front of the vehicle. If the primary path of the vehicle is blocked, it will seek an alternative path; however, if both paths appear to be blocked, the servo motors and vehicle operations will come to a halt and flash an LED light. Assuming the obstacle is not removed, the LED light will continue to flash, informing the appropriate team will investigate the issue.

Literature Review

While our team has not worked directly in the field of autonomous technology, we believe that the closest project to ours is the fully autonomous vehicles that are being driven across the country. General Motors, a well-known company that develops autonomous vehicles themselves, believes that these vehicles are the future and are going to positively change people's lives forever. When it comes to the technology that they have already developed for autonomous vehicles, they have released Super Cruise and Ultra Cruise which is already being implemented in vehicles in 2023. Specifically, General Motors stated, "Super Cruise is the industry's first true hands-free driver assistance technology that enables drivers to travel hands-free on more than 400,000 miles of compatible roads. Ultra Cruise will offer a destination-to-destination hands-free driving experience, designed to ultimately enable hands-free driving in 95 percent of all driving scenarios (*Zero Congestion With Self-driving Vehicles | General Motors*, n.d.)". While these technologies have great benefits and functionalities when it comes to the autonomous space, there is still a small percentage of driving scenarios that are not accounted for and may result in accidents and potentially deaths depending on the scenario the driver is placed

in. Looking at our current generation and era of technology, it is able to make great accomplishments and milestones, but our team believes that there are many variables that can affect how these technologies operate, resulting in the 95% hands-free in all driving scenarios opposed to the theoretical 100% that customers are wanting to see. However, because our autonomous Arduino vehicle's factors are operating in an indoor environment and can adjust the conditions that our vehicle is placed in, we are confident that our vehicle will successfully serve its intended purpose. Additionally, we are confident that our product is affordable compared to General Motors and other automotive manufacturers. General Motors has announced investments in EV and autonomous vehicle (AV) development of more than $35 billion from 2020 through 2025 and aims to work with federal, state, and local governments on "implementation of supportive policies to move the United States closer to carbon neutrality consistent with the goals of the Paris Agreement (Simhoni, 2023)". While our group has not invested millions of dollars unlike other major companies that are making a world-wide product, we certainly have fewer external factors to be concerned about, allowing our product to act completely functional with minimal errors being encountered. It's apparent that autonomous vehicles are being supported and that organizations believe in the potential they can serve our society. Additionally, this makes our product affordable and feasible for the buyers who are interested because of the lower cost associated with a project of this caliber.

Autonomous Vehicle Functional Requirements

Our team made sure to define our functional requirements carefully, so that we could decide the type of hardware, software, and other required assets for the autonomous vehicle. Here is our full list of functional requirements that our vehicle performs.

Autonomous Delivery Vehicle Functional Requirements:

- The vehicle should be able to pick up packages and securely hold onto them without dropping contents.
- The vehicle should be able to place down packages in a pre-determined area (varies depending on the vehicle's current objective/lap).
- If there is an unaccounted object placed in front of the vehicle, it should take an alternative path, if available. Otherwise, halt all vehicle operations and check if the obstructive object continues to exist. If so, enable the LED alarm and await assistance. Otherwise, continue vehicle operations.
- The vehicle should successfully follow the black-lined path by default, unless the vehicle's light sensor detects a color (such as red, yellow, orange, or purple). From here, the vehicle will perform its designated action and return to following the black path.

As mentioned previously, our team would like to draw attention to the significance and importance of the technology that can potentially be implemented in various industries. Also, these technological devices have the capability of completing simple tasks that would not need to be performed by a human-being, allowing businesses to become more efficient from a resource and time perspective.

Autonomous Vehicle Design

Elaborating on the design aspect of our vehicle, our team has collaborated closely with one another to ensure that our functional requirements were realistic and accomplished the functionality that our team sought for. We understood that our vehicle was incorporating various types of hardware onto a singular vehicle and wanted to ensure that the vehicle was safe, reliable, and practical when it came to bringing these technologies together. When discussing the safety aspect of our vehicle, it is important to recognize that our vehicle is designed to work in an

indoor warehouse environment and understand that various teams throughout the company are going to be occupying any given workspace. Specifically, we needed to design the vehicle, so that employees are safe, which was our top priority when planning the design aspect.

We've incorporated safety measures by ensuring that all operations come to a halt when the vehicle detects an obstructive object (varies) using our ultrasonic sensors that are placed in the front of the vehicle. When discussing reliability, our group understood that the RGB light sensors may fluctuate and change frequently, especially when the vehicle shakes and rumbles as it's moving. Because we understood the importance of keeping our results consistent when viewing colors, we decided to calibrate our RGB sensor within the code itself to ensure that we're obtaining the correct values.

Discussing more about the creation of the vehicle itself, our team found that some materials that allowed the package to be secured were fragile and broke easily and therefore did not believe that it was ethical to produce this product knowing this information. Because we understood the flaws and were able to locate the source of the problem, we were able to create an identical piece that was more robust and rectified the problem. Our team aims to follow the National Society of Professional Engineers: Code of Ethics and aims to follow their values.

Hardware for Autonomous Vehicle

Understanding how autonomous vehicles operate is a formidable challenge. It involves more than just controlling motors or obeying traffic signals which involves selecting the appropriate hardware that allows the vehicle to move precisely as directed, with specific timing, enabling it to stop, start, and navigate while evading obstacles. This is crucial to prevent damage to the vehicle's surroundings and, more critically, to ensure the safety of workers in proximity to these driverless vehicles. Our primary objective is to prevent such incidents by utilizing highly

sophisticated hardware capable of safely guiding the vehicle to its designated destination on a routine basis. While there are already vehicle types used in warehouse logistics, our aim is to identify more cost-effective hardware solutions compared to the vehicles typically used in warehouse environments, which can often cost hundreds of thousands of dollars. Our aim is to construct a scaled-down replica of a warehouse setting using the 'LAFVIN Mechanical 4WD Robot Arm Smart Car Kit Robot' purchased from Amazon at a cost of approximately $49.99 as shown in figure1 (Amazon.com, n.d.).

LAFVIN Smart Car Robot

This kit offers various advantages for our project, featuring an Arduino Uno board replica at its core, enabling comprehensive control over motor functions, robot arm movements, and sensor data collection, including road tracking and ultrasonic sensors essential for our driverless vehicle development as shown in figure 1 with some of the parts that is included in this package. Additionally, we have acquired foam board materials to recreate the warehouse floor and shelving units, providing a realistic environment. Ultimately, our goal is to harness the versatility of this smart car kit to create a functioning miniature warehouse environment, complete with autonomous vehicle capabilities and lifelike surroundings, facilitating thorough research and cost-effective experimentation.

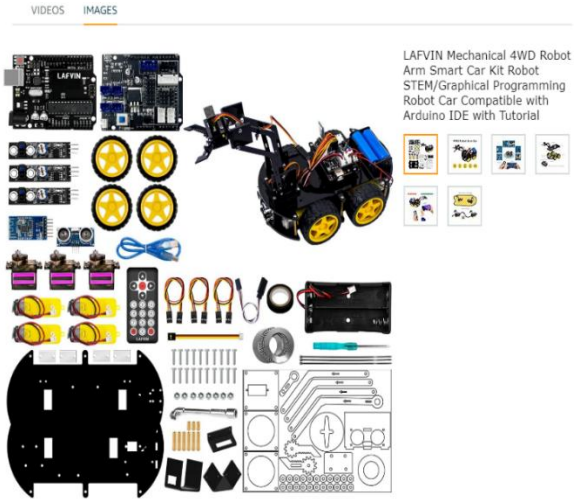


Figure 1: LAFVIN smart car robot bought from Amazon.com

We have chosen the Arduino microcontroller over other options due to its exceptional versatility in controlling motors, servos, LEDs, and interfacing with various sensors, which are

crucial for managing our driverless vehicle. While there are more expensive microcontroller alternatives available, Arduino boards offer a cost-effective solution, with genuine Arduino boards priced at approximately $26.50 on Amazon.com, and even more affordable options available for generic ones, in contrast to the Raspberry Pi 4, which costs around $98.00 on Amazon.com. The Arduino Uno, a member of the Arduino family, is equipped with an Atmega328P microchip with a processing power of about 16MHz, which may be considered relatively slow when compared to the Raspberry Pi (Szczepaniak, 2023). However, choosing Arduino over Raspberry Pi provides several advantages beyond cost-effectiveness. Arduino is an open-source platform, supporting programming in C, C++, or Python, and offering the flexibility to connect multiple sensors or shield boards to meet our project's requirements. Furthermore, the Arduino Uno is conveniently included in the LAFVIN kit, eliminating the need for additional board purchases. It is worth noting that one limitation of the Arduino Uno board is its 20 pins available for signal inputs or outputs, with 14 of these (D0-D13) dedicated to digital signals and the remaining 6 (A0-A5) capable of controlling analog signals, which can also be used as digital pins. While this pin count is sufficient for a limited number of sensors, some sensors may require multiple pins, while others can operate with just one pin, necessitating careful planning for our project.

LAFVIN Pin Allocations

The LAFVIN smart car kit has already allocated most of the available pins, and our project may necessitate the addition of more sensors to achieve the desired safety and functionality required for a driverless car. This predicament could potentially lead to the removal of some sensors from the smart car, which poses a challenge since each of these sensors is integral to our project. However, Arduino boards offer a practical solution to this dilemma

through the incorporation of the I2C communication protocol, also known as 'I squared C.' With this communication protocol, we can effectively control up to 128 devices using just two of the 20 available pins where 'SDA' (serial data pin) is for sending and receiving data and 'SCL' (serial clock pin) for managing the clock signal between devices on the I2C bus with each sensor hooked up in parallel to these two pins (I2C tutorial, n.d.). This feature proves invaluable as we integrate additional sensors into our customized driverless smart car. The I2C protocol operates by assigning a unique address to each sensor in hexadecimal format, such as '0x01,' all managed through C++ code within the Arduino IDE platform.

Nevertheless, it is important to note that some sensors come with predefined addresses, which can lead to conflicts when attempting to install multiple sensors. For instance, the RGB color sensor we utilize for color detection employs the '0x29' address. When we attempted to incorporate another sensor for distance detection using the I2C protocol, it coincidentally used the same '0x29' address, resulting in address conflicts. Consequently, we had to forgo the distance sensor in favor of prioritizing the RGB color sensor. As engineers, our primary focus is to maximize the integration of hardware to ensure control, safety, and overall functionality in real-world applications. However, the limited number of available pins presents challenges when incorporating multiple sensors, requiring meticulous planning and problem-solving in our equipment development process.

CD74HC4067 and TCA9548A Multiplexers

To address the limitations of available pins on the Arduino board, we explored alternative methods of expanding our pin capacity by incorporating external boards to serve as pin extensions. One of the solutions we considered was the use of multiplexers, a common approach in Arduino projects. We obtained two types of multiplexers: an analog multiplexer, namely the 'CD74HC4067' as shown in figure 2, which utilizes six pins from the Arduino board while providing 16 additional pins. This effectively granted us 12 extra pins for controlling other sensors, offering a favorable outcome.

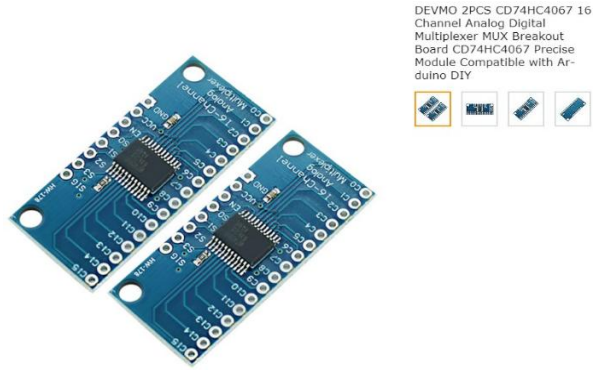


Figure 2: CD74HC4067 analog multiplexer to allocate more pins

The second type of multiplexer employed the I2C protocol known as 'TCA9548A' that presented a more efficient option compared to the analog counterpart which is shown in figure 3. It required only two pins for operation while yielding additional pins, totaling 16. This seemed promising, and we were eager to implement at least one of these solutions.

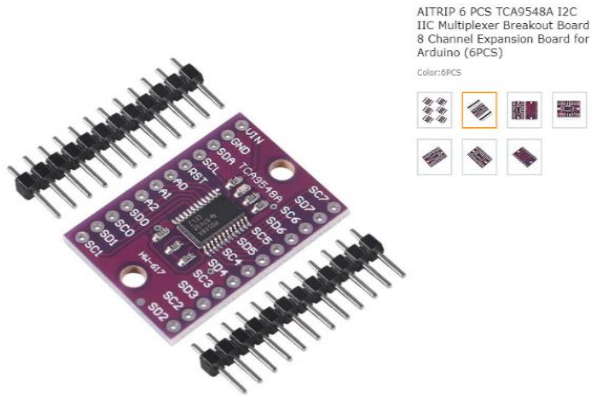


Figure 3: TCA9548A multiplexer to add more pins to microcontroller

However, we encountered a challenge not related to the multiplexers themselves but rather to the fundamental operation of any multiplexer. A multiplexer functions as a switch that directs one signal, such as an output from the Arduino to a single receiver, typically a connected sensor. This means that only one sensor can receive the signal at a given moment, making it

impossible to have two or more sensors connected to the same multiplexer simultaneously receiving or sending signals. It's essential to note that these multiplexers can function as either Multiplexers 'MUX' or demultiplexers 'DMUX.' While they can output signals to sensors as a MUX, they can also receive signals from multiple sensors as a DMUX. This resulted in you either make pins as all outputs to be set as a multiplexer, or you can make all pins inputs in a DMUX settings.

Unfortunately, this posed an issue for our project since some sensors, like the ultrasonic and RGB sensors dedicated multiple input and out pins, must continuously transmit signals to the controller (Arduino) without interruption due to safety concerns that may arise if any signal were interrupted during vehicle operation. After extensive research for a more suitable solution, we discovered a shield board that could effectively transfer pins from Servo motors. This discovery allowed us to add three pins from these Servo motors to be removed from the main microcontroller. Fortunately, we found the 'PCA9685' PWM Servo Motor Driver board, which utilizes the I2C protocol, requiring only two pins for operation, as previously mentioned. This discovery was a game-changer for our project, transforming it from a potentially flawed setup into a more robust and adaptable system capable of meeting our operational needs.

PCA9685 Servo Motor Driver Board

Originally developed by Adafruit, the 'PCA9685' addresses the pin limitations inherent in microcontrollers like Arduino by utilizing PWM (pulse-width modulation), a common technique employed for controlling servos and LED lights as analog devices as shown in figure 4 installed

on the vehicle with outputs goes to servo motors and the other goes to KY-034 LED color sensor. This module offers the flexibility to control digital sensors, allowing us to specify the maximum number of steps (up to 4096) to adjust the PWM and achieve a duty cycle of either 100% for full output or 0% to turn it off, a feature we utilized to activate and deactivate the LED alarm module in our project. The servo driver board provides 16 channels (0-15), simplifying the mapping of pins to specific sensors or servos. Integration with this board was facilitated through the Adafruit library, ensuring seamless connectivity. Additionally, it's worth noting that multiple boards can be daisy-chained together, with the potential to connect up to 62 boards, resulting in an impressive 992 output pins, a valuable resource for more complex projects. However, for the purposes of our project, a single servo driver motor board sufficed, allowing us to achieve our project objectives effectively.

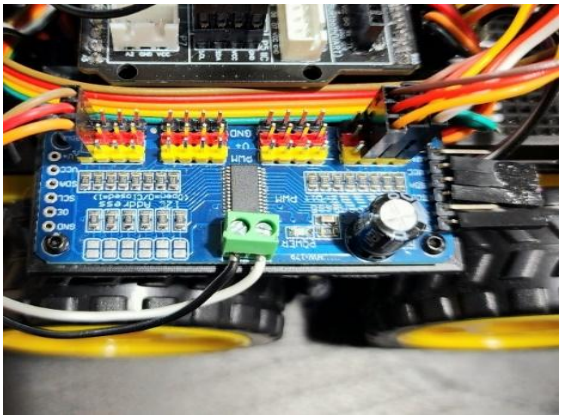


Figure 4: PCA9685 servo motor driver board helps reallocate pins in microcontroller

KY-034 7-Color LED Sensor

'PCA9685' servo motor driver board played a significant role in alleviating the pin constraints on the Arduino board by enabling us to offload the control of the robot arm's servo motors onto this dedicated board. This allowed us to free up pins on the Arduino for additional sensors and functionality. Specifically, we added an extra 'HC-SR04' ultrasonic sensor to the side

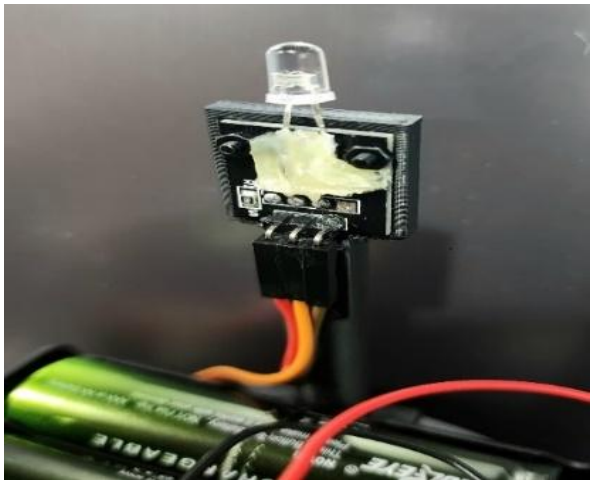


Figure 5: Using KY-034 LED light sensor as alarm system when obstacles occur

of the vehicle to detect nearby objects, aiding in the object detection process. Furthermore, we incorporated a 'KY-034' sensor, utilizing a 7-color LED flashing module to indicate alarms in the event of any issues during the vehicle's operation, such as halting when an object is in front of the vehicle as shown in figure 5 installed on 3D printed pole. The servo motor board provided us with a total of 16 pins, three of which were allocated to control the robot arm's servo motors, while one was used for the LED flashing module.

HR-SR04 Ultrasonic Sensor

One of the most critical aspects of this project revolves around safety concerns, particularly in preventing damage or injuries during vehicle operation. This is where the ultrasonic sensor assumes a pivotal role, safeguarding bystanders who may be in the vehicle's path. Ultrasonic sensors are widely used worldwide and are even integrated into modern vehicles, typically seen as sensors on the rear or front bumpers. These sensors provide proximity warnings to drivers, alerting them to nearby objects through an audible alarm, enhancing situational awareness even when objects are not directly visible.

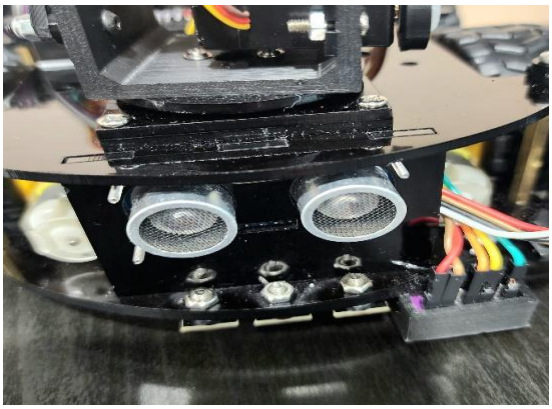


Figure 6: HC-SR04 ultrasonic sensor is used to obstacle avoidance

In our project, the LAFVIN smart car is equipped with an 'HC-SR04' ultrasonic sensor positioned at the front as shown in figure 6. The HC-SR04 ultrasonic sensor necessitates four wire connections: two for positive and negative power, and the other two for trigger and echo signals. The sensor operates by emitting ultrasonic pulses from the trigger pin, with each pulse lasting only 10 microseconds when the pin is set to a high state. This initiates an ultrasonic burst of eight pulses at 40KHz (Engineers, 2023). The 'Echo' pin remains in a high state until the

sensor detects and receives an echo, at which point it returns to a low state (Engineers, 2023). This time interval is used to calculate the distance in the application software.

Typically, ultrasonic sensors can measure distances ranging from 3cm to 400cm in an open field (*Ping Ultrasonic Range Finder | Arduino Documentation*, n.d.). This distance measurement system plays a pivotal role in determining the location of objects in front of the vehicle, enabling the implementation of specific conditions during vehicle movement. For instance, if the vehicle detects an object within 10cm in front of it, it must come to a complete stop until the obstacle is removed. This feature significantly enhances the safety of employees and objects in the vehicle's path during operation. Additionally, integrating the same ultrasonic sensor on the side of the vehicle assists in identifying objects during item pickup and drop-off at designated locations. This information guides us in determining whether a station requires item handling or is empty, allowing for additional condition-based programming. These sensors are instrumental in ensuring the safety and efficiency of our project.

TCRT5000 Infrared Signal Reflective Tracking Sensor

Another pivotal sensor that assumes the role of guiding the vehicle to its predetermined destination is the 'TCRT5000' infrared signal reflective tracking sensor, and our LAFVIN smart car comes equipped with three of these sensors as standard which is shown in figure 7. The TCRT5000 employs an Infrared Signal Reflective Track Sensor to enable our autonomous vehicle to follow the black lines we designate on its path.

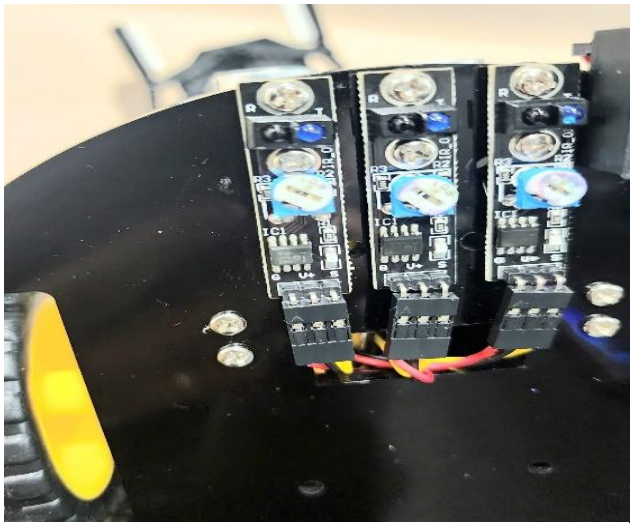


Figure 7: TCRT5000 IR sensors are used as tracking mechanism

The TCRT5000 infrared reflective sensor offers several advantages and proves to be a valuable addition to our overall product range. These reflective sensors are designed with an infrared emitter and a phototransistor within a leaded package that effectively blocks visible light (Vishay, n.d.). The infrared emitter emits light that is invisible to the human eye but detectable by electronics and other devices. The phototransistor is responsible for detecting the infrared light and transmitting a signal to a computer or another device. These three sensors are positioned on the bottom front of the vehicle. Their functionality is achieved through programming, with the middle sensor sending an active high signal to the Arduino board while the vehicle moves along the black line, while the other two sensors on the right and left of the middle sensor function as active low signals sent to the microcontroller. If, for any reason, one of the sensors on the sides crosses over the black line, it activates a high signal, prompting the vehicle to adjust its course either to the right or left, depending on the situation where one of these sensors crosses the black line. This method ensures that the smart car moves in a straight line by accurately tracking the black line, maintaining consistency in its movement throughout the tracking process.

One issue we encountered was when the vehicle completely bypassed the black line, signifying that none of these sensors were following it, which could potentially lead to unforeseen damage to the vehicle and its surroundings. To address this concern, we implemented a condition that triggers an immediate stop of the vehicle and alerts the controller if it veers off the tracking lane. While the option of incorporating six tracking sensors, with three in the front and three in the rear, might seem feasible, it would impose additional pin requirements that our smart vehicle lacks. Given the considerations of our project, working solely with tracking sensors in front of the vehicle proves sufficient to accommodate this condition and maintain the

vehicle's stability in following the tracking lane, represented by the black line.

Choosing the Right Sensor for Picking and Dropping Stations

The core of this project is finding a way to apply a certain type of sensor for picking up or dropping off from one station to the other without the intervention of humans through the use of driverless vehicles. Our plan is to determine the type of sensor to be used during the pickup or drop-off operation at each station, and we have chosen three of these sensors.

The first one is the 'TCRT5000' IR sensor, which can detect light and dark and was used as a tracking lane while the robot is in a driving state, which proved to be interesting. However, due to the placement of both tracking lane sensors and the addition of an additional tracking sensor for pickup and drop-off on the same road, conflicts arose while driving. The vehicle could mistakenly perceive another line as a driving lane, potentially causing harm to its surroundings. Another sensor considered was the 'HC-SR04' ultrasonic sensor, which was a great idea for detecting objects. However, during its implementation, issues arose with its sensitivity to the surroundings. It detected not only the specific objects to be picked up or dropped off but also walls and even bystanders. This led to inaccuracies in determining the vehicle's exact location during pickup or drop-off.

An alternative option was to use a vision sensor that could locate and identify any objects necessary for the logistics operations. However, after researching the prices of such sensors, it became evident that they would significantly exceed our low budget. The lowest-priced sensor we found was around $66, which was more expensive than the smart car itself. Consequently, we opted for a different approach that would be more cost-effective. After thorough research, we decided to purchase the 'TCS34725' RGB color sensor, priced at around $10.88 on Amazon.com. This sensor was a better fit for our budget and had a proven track record of applications in

various industries, such as sorting objects in industrial settings and the robotics industry. It turned out to be an excellent addition to our project, making our driverless vehicle a more sophisticated solution without exceeding our planned budget.

TCS34725 RGB Color Recognition Sensor

The TCS34725 RGB color recognition sensor, manufactured by TAOS and designed by Adafruit, serves as the cornerstone of this entire project as shown in figure 8 assembled on the smart vehicle inside an enclosure. This sensor possesses the capability to detect colors accurately, which aligns perfectly with our project's objective of establishing pickup and drop-off conditions based on color-coded stations within a warehouse environment. This sensor forms the foundation of our project, enabling us to differentiate between colors on the road's surface. When the RGB color sensor crosses over the floor, it can identify colors like blue, yellow, or green, and subsequently halt and collect objects destined for transfer to another station for appropriate drop-off.

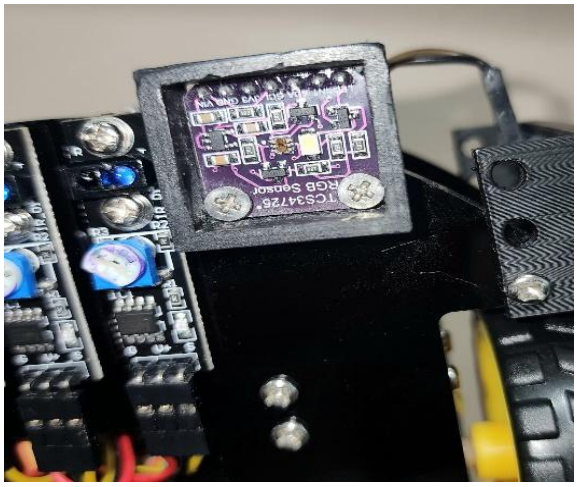


Figure 8: TCS34725 RGB color detection sensor, played the main role in our project

The structural operation of the RGB color sensor relies on photodiodes that emit light and then measure the intensity of reflected light to determine the object's color whether it is a blue, green, white, or any other. The resulting filtered colors are converted into currents, which are then read as a 16-bit digital value (TAOS, 2012). Additionally, the TCA34725 operates with the I2C protocol, a beneficial feature that minimizes the number of wires required for connection to the microcontroller.

During the installation of the TCA34725 sensor onto the smart car, we encountered several challenges. The most significant issue arose with color recognition, as the received data demonstrated erratic fluctuations, depicting readings of red, green, and blue (RGB) colors unreliable. This setback prompted us to think whether to continue using this sensor or explore potential solutions through more in-depth research. After consulting various websites and searching various YouTube videos, we arrived at the conclusion that such sensors might require calibration for reasonably accurate but not entirely precise results (Tech with Mike, 2019). To accomplish this, we placed a black color close to the sensor to obtain its RGB color reading, which should ideally be 0,0,0 for black. Any readings above this baseline, for instance, 10,25,30, were adjusted to 0,0,0 through mapping and constraining these values in the software application. A similar approach was applied to white, which normally registers as 255,255,255, with any values exceeding this range constrained to the maximum white values. This calibration significantly reduced the fluctuations in the RGB color values but proved insufficient to rely upon entirely, as factors such as the car's speed and ambient lighting continued to influence the values.

To address these issues, we first slowed down the vehicle, maintaining a steady speed, which resulted in a more consistent range of RGB color values. Second, we introduced a threshold for each RGB color value. For instance, when using purple, the values fell within the ranges of 135-195 for red, 75-98 for green, and 115-151 for blue. These minimum and maximum value thresholds were incorporated into the coding, further mitigating value fluctuations.

Second, to combat disruptions caused by varying lighting conditions, we enclosed the RGB color sensor in a 3D-printed enclosure, positioning it as close as possible to the designated color-coded distribution stations. This step significantly reduced the fluctuations in RGB color

values, thus addressing timing concerns during vehicle operation. Furthermore, we retained the ultrasonic sensor on the vehicle's side, integrating it with the RGB color sensor. These sensors work mutually for a better logistics distribution, with the ultrasonic sensor determining the presence of objects at the designated station and the RGB color sensor verifying that it is indeed the correct station for either pickup or drop-off. This combined approach proved to be a safe and efficient solution, enhancing the overall operation. In conclusion, the TCA34725 RGB color recognition sensor represents a valuable tool with the potential to open new opportunities for organizations seeking to implement autonomous vehicles within their operations.

HC-05 Bluetooth communications

In the context of using a Python control center application with an Arduino microcontroller on a driverless vehicle, effective communication is vital for sending and receiving commands wirelessly. Initially, we attempted to utilize the JDY-16 BLE Bluetooth device that came with the LAFVIN smart car kit. However, our search for information on how to integrate the JDY-16 Bluetooth device into the Python application was challenging. We struggled to find suitable libraries and guidance to establish a successful connection, eventually leading us to abandon the JDY-16 Bluetooth device.

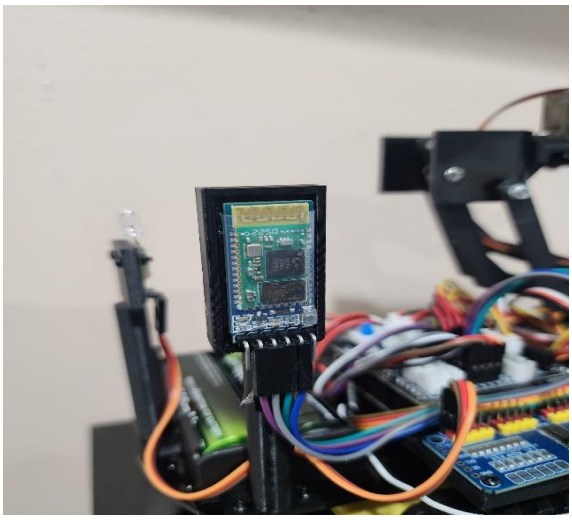


Figure 9: HC-05 is the central communication module that connected the both programs

After extensive research, including watching various YouTube videos, we found that the "HC-05" Bluetooth module was a more feasible communication solution as shown in figure 9. There were numerous resources available explaining how to configure the HC-05 device and

which libraries to use on both the Python and Arduino sides. In particular, we benefited from a YouTube video by ChhayLy, which explained how the HC-05 Bluetooth module facilitates communication between Python and Arduino (ChhayLy, 2020). In the video, ChhayLy demonstrated the process of identifying the serial port associated with the paired HC-05 device on a Windows PC or other computer. Additionally, we learned that installing the “Pserial” library was crucial for the Python application's functionality. On the Arduino side, we encountered challenges as the HC-05 Bluetooth module typically uses the TX (D1) and RX (D0) pins for data transmission and reception. These pins also serve as the USB serial port communication interface with the Arduino microcontroller. To overcome this conflict, ChhayLy's video recommended implementing a software serial library, allowing us to use alternative pins for communication and thus achieving a more stable connection between Python and Arduino.

While Bluetooth is a suitable choice for short-range communication, it may not be ideal for environments requiring longer-range coverage, such as warehouses. Bluetooth's range is limited to approximately 10 meters, which may fall short of the control center's reach. An alternative recommendation would be the use of IR radio transceivers, which can extend communication ranges to over 1 kilometer on some devices. Alternatively, devices equipped with Wi-Fi connectivity could offer an even broader communication interface. However, for our project's requirements, the HC-05 Bluetooth module proved capable of meeting the communication needs within a shorter range.

3D Printing with TinkerCAD

Another problem that arose during the project's progress was related to the vehicle's components, which proved to be very brittle and prone to breakage if mishandled. This was particularly true for the robot arm, which consumed a significant amount of our time as we searched for spare parts. Our attempts to find replacement parts on platforms like Amazon and Google yielded no results, and the only available option seemed to be purchasing a brand new LAFVIN smart robot car, which would have further strained the project's budget. Fortunately, one of our team members possessed a 3D printer, which turned out to be a lifesaver for replicating the broken vehicle parts. However, a new challenge emerged as 3D printing required expertise in 3D CAD design, an area where none of our team members had prior knowledge.

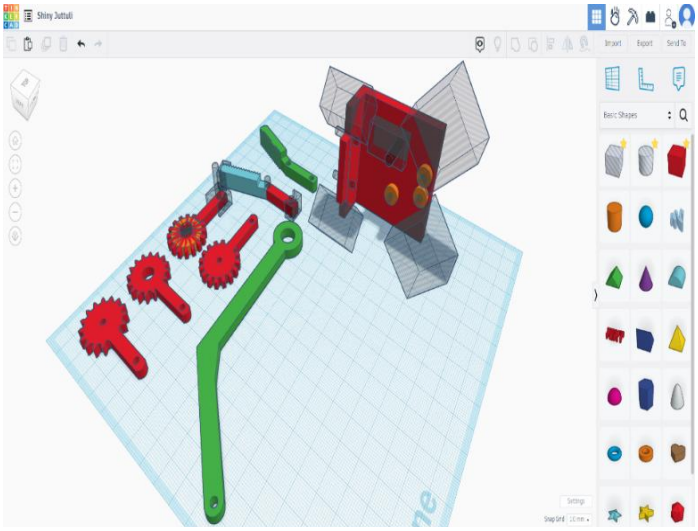


Figure 10: Using TinkerCAD are the main design for the 3D printed parts

To address this issue, we decided to start learning how to design these complex parts. We chose TinkerCAD, a free platform known for its user-friendly interface as shown in figure 10 showing multiple parts were built to replace or add parts to the vehicle structure. An essential aspect of 3D CAD printing is ensuring precise and accurate measurements of the product.

Without accurate measurements, issues may arise during the reassembly of the parts. To achieve this level of accuracy, we relied on multiple tools, including a caliper for measuring hole sizes, width, and height of each part. We also used a drill driver to create small holes where the

3D printer lacked precision, and also used dial round ruler to measure the angles of certain parts. Despite its simplicity, TinkerCAD proved to be a powerful tool for beginners and intermediate learners, enabling us to design and create our own products from scratch using the platform's limited set of tools. This highlighted the importance of precision and accuracy in achieving favorable outcomes which is shown in figure 11 which shows the designing and assembling claw grippers that required a precise gear ratio to move the gripper arms freely.

Figure 11: Claw are made by the 3D printer to accommodate a wider angle for usability

Working with TinkerCAD was not particularly difficult, but it was tedious and time-consuming. The need for precise measurements and the waiting time for 3D printing added to the challenge. Any inaccuracies in our measurements or malfunctions in the 3D printer during the printing process set us back in terms of meeting our project's deadline. While this aspect of the project was not initially planned, it taught us valuable lessons about handling unexpected challenges. It underscored the fact that not all projects proceed smoothly, and unforeseen obstacles can arise. This experience has shaped our approach to handling hiccups during this project and any future ones that may come our way.

Hardware Recommendations

Some recommendations that could significantly advance our project, assuming time and budget constraints are not an issue:

- Vision Sensor: Incorporating a high-quality vision sensor equipped with object recognition capabilities can significantly enhance the precision and accuracy of object detection. Look for sensors with robust image processing capabilities capable of identifying objects, obstacles, and lane markings.
- Pressure Sensor for Claw: Integrating a pressure sensor into the claw mechanism can provide an additional layer of safety and improve object detection during the pickup and drop-off of items. This sensor can ensure that the claw securely grasps objects and can promptly detect any issues during handling.
- Additional Ultrasonic or Vision Sensors: Expanding the sensor array to enhance environmental awareness and human interaction detection is a wise move. Consider adding more ultrasonic sensors or high-resolution vision sensors to cover blind spots and bolster the vehicle's collision avoidance capabilities.
- Improved Communication Module: Upgrading to a more robust and longer-range communication module, such as the NRF24L01 or a similar solution, can extend the operational range of the driverless vehicle within the warehouse. This enhancement can significantly improve its versatility and utility in larger workspaces.
- Speed Sensor: Integrating a speed sensor can provide precise control over the vehicle's speed, enhancing safety and enabling adaptation to changing conditions within the warehouse environment.
- Enhanced Microcontroller: Consider upgrading to a more powerful microcontroller, like the Raspberry Pi, to provide additional processing power and memory. This can accommodate larger and more complex applications and algorithms, expanding the vehicle's capabilities.

- Battery and Power Management: Implementing a dedicated Battery and Power Management system is essential to ensure the vehicle's reliability and efficiency. This includes integrating a high-capacity battery and a well-designed power system to extend operational uptime and minimize downtime for recharging.

These recommendations should be considered for future projects, potentially serving as valuable additions to real-world endeavors that our team aims to undertake in the future.

Vehicle Hardware Summary

Our project initiated the ambitious task of developing a driverless smart car for warehouse logistics, aiming to enhance efficiency and safety while staying within a tight budget. We faced various challenges along the way, from pin limitations to sensor integration and 3D printing complexities. These challenges, however, pushed us to think innovatively, adapt to unexpected situations, and make ideal decisions. We utilized versatile hardware solutions, such as the Arduino microcontroller and the 'PCA9685' servo motor driver board, to overcome pin constraints and expand our sensor capabilities. The integration of sensors like the TCRT5000 for tracking and the HC-SR04 for proximity detection enabled us to navigate with precision and ensure safety. Additionally, the TCA34725 RGB color sensor proved to be a cost-effective yet powerful choice for color recognition. Our experience with 3D printing and CAD design emphasized the importance of precision and adaptability in engineering projects. In the end, our project demonstrated the potential for autonomous vehicles in logistics and provided valuable insights into overcoming challenges that may arise in such careers.

Software Development and Implementation

The development and implementation of the autonomous vehicle control system encompass two crucial components, each serving a distinct role in coordinating the operation of

the vehicle. First and foremost, the Arduino IDE platform assumes the role of the backend powerhouse through the use of C++ programming language. This backend code manages the vehicle's hardware, administering critical functions and ensuring precise execution. On the other hand, involves the utilization of Visual Studio Code, serving as the front-end interface for the user as applied in Python language. This application will handle control and issue commands to the autonomous vehicle. Using Bluetooth serial communication will establish reliable channels between the front-end command center and the backend hardware infrastructure. This suitable interaction between the frontend and backend components forms the backbone of a sophisticated autonomous vehicle control system, guiding in a new era of automated precision.

Front-End Python Application

The Python application serves as the control center, facilitating seamless command transmission through Bluetooth serial communication. To establish this vital connection, a computer initiates pairing with the embedded Bluetooth module in the driverless vehicle. Once this connection is successfully established, the Python application gains the capability to dispatch commands to the Arduino microcontroller residing within the vehicle. These commands are transmitted as alphanumeric characters from Python to the Arduino controller, with each character serving as a directive for a specific operation executed on the Arduino side. The Python application streamlines this interaction through a user-friendly interface, allowing users to trigger operations by simply clicking

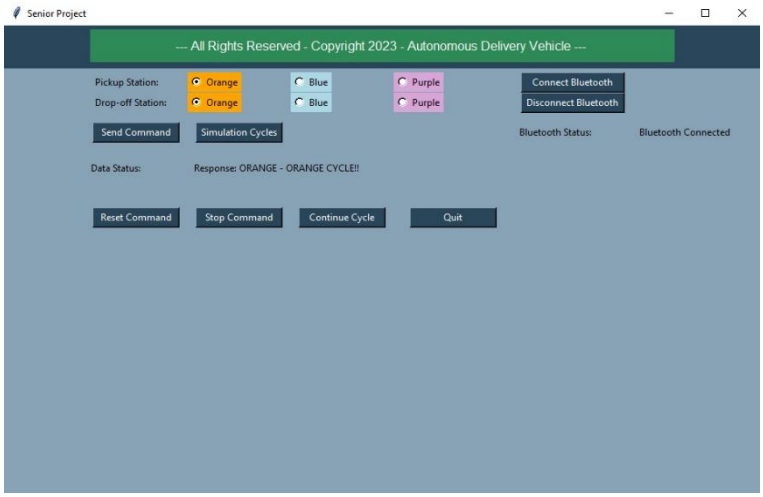


Figure 12: Python application used as front-end platform to control the driverless vehicle

buttons. These buttons, when activated or deactivated, correspond to various functions within the vehicle's control system. Moreover, the Python application prompts users to input a COM port, a crucial step for establishing a secure connection between the Bluetooth devices. For added transparency, each command sent from Python prompts a status report detailing the ongoing operation within the vehicle system. This real-time feedback enables the controller to monitor operations and respond promptly in case of any anomalies. Python uses the Tkinter GUI library to create an intuitive user interface, simplifying the control process.

Despite its straightforward design, this application empowers users to effectively manage and oversee the operations of the driverless vehicle, particularly in a demanding logistics distribution environment. Figure 12 shows the main platform of the autonomous delivery vehicle application which contains multiple command buttons to control and handle the operation of the vehicle. Also, there are two buttons added to establish the connections between the Bluetooth serial communication. Moreover, there are two status labels that will show if the connection has been established or not, and also shows the status of the operation after sending command to the vehicle. A small popup window to add the desired Bluetooth COM port to establish the communication between PC and driverless vehicle as shown in figure 13. This is detrimental when it comes to connecting the computer to the vehicle.

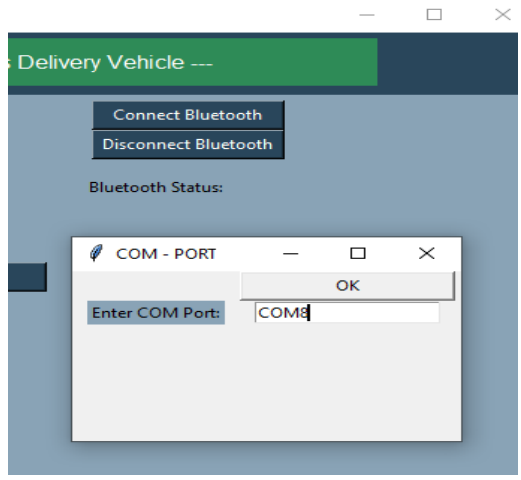


Figure 13: Using a pop-up window to insert the serial communication port

Code Explanation

Our team will be sure to elaborate on the most important aspects that allow our vehicle to

operate and while we are also uploading our finalized code, our goal is to explain and discuss *how* the vehicle operates without too many technical details, allowing users to understand the role the various functions play throughout. Let's start by visiting the main sections.

Python Code

In this small portion of the code will show a small pop window to insert a text for the COMP port to be used as serial connection between Bluetooth devices in the function of get_com_port().

```
def get_com_port():

    # Small popup window to insert the COM port into text field
    com_window = tk.Toplevel(window)
    com_window.geometry('{}x{}'.format(260, 150))
    com_window.title("COM - PORT")

    com_port_label = tk.Label(com_window, text="Enter COM Port:", bg='#89A3B6')
    com_port_label.grid(row=4, column=1, padx=10, sticky='w')
    com_port_entry = tk.Entry(com_window, bg="white")
    com_port_entry.grid(row=4, column=2, padx=10, sticky='w')
```

This part of the code inside get com port function will establish the connection through the use of serial library com_port "COM1, etc." with baud rate of 9600 baud rate which must match the Bluetooth on Arduino side and timeout of 1 second give or take to establish connection and communicate with other side.

```
        try:
            #This is the connection to the serial connection with Bluetooth
on the Arduino side
            ser = serial.Serial(com_port, 9600, timeout=1)
            btooth_label.config(text="Bluetooth Connected")
            com_window.destroy()  # Close the COM port input window
        except serial.SerialException as e:
            btooth_label.config(text=f"Bluetooth Connection Error: {str(e)}")
```

Another important part of the Python code that we needed to add is threading. The issue we encountered was that we weren't able to retrieve data continuously. Ideally, it should retrieve multiple data points as long as the car is in motion or stationary, sending status updates back to the Python

```
#This method to retrive data from the Arduino side while the vehicle is running.
def retrieveData():
    global ser
    try:
        if ser is not None and ser.is_open:
            #retrieve the data from the arduino side as readline from serial port "ser"
            data = ser.readline().decode('utf-8')
            if data:
                data_label.config(text=f"Response: {data}")
    except serial.SerialException as e:
        data_label.config(text=f"Serial Error: {str(e)}")  # Return the error message
```

controller station. Thanks to a NeuralNine YouTube video, we were able to address this problem by implementing a separate thread dedicated to the retrieving of data from the Arduino side (NeuralNine, 2023).

The code essentially uses a separate thread called ‘data_thread’ that runs concurrently with the ‘window.loop()’ main thread. We set the daemon attribute to true so that when ‘window.loop()’ is closed, the ‘data_thread’ will also close at the same time. This approach helped us solve the issue of data retrieval.

This method is to send command to the Arduino side by using command equal to a text which is a character in this condition of our application and then using serial write command with command. Encode() to send this message which are sent as ASCII characters only, however, if you need to use other characters than ASCII characters you must include utf-8 inside the parentheses of the encode such as ser.write(command.encode(‘utf-8’), so that way can multitype of characters.

```
#defention to send the multi colored option to Arduino
def send_command(sending_color):
    try:

        command = f"{sending_color}"
        ser.write(command.encode())

        time.sleep(0.1)  # Adjust the delay as needed
        data = retrieveData()
        data_label.config(text=f"Response: {data}")
    except serial.SerialException as e:
        data_label.config(text=f"Serial Error: {str(e)}")
```

This method will take actions from the radio button assigned as different colors with specific station of the pickup and drop of each color. This function will combine the pickup and drop-off colors to make it as one character and will be sent to Arduino side action of operation as such when trying to pick from orange station and drop it

```
def get_pick_drop():
    #getting the values of each radio button
    pick_color = pickup_color_var.get()
    drop_color = dropoff_color_var.get()

    #this is just to check values of the radio buttons in console
    print("Pickup Color:", pick_color)
    print("Drop-off Color:", drop_color)

    # Initialize sending_color with a default value
    if pick_color == 'o' and drop_color == 'r':
        sending_color = 'o'
    elif pick_color == 'o' and drop_color == 'l':
        sending_color = 'b'
    elif pick_color == 'o' and drop_color == 'u':
        sending_color = 'p'
    elif pick_color == 'b' and drop_color == 'r':
        sending_color = 'l'
```

off at blue station as an example. This way will ease the number of characters sent through Bluetooth communication service.

This portion of snippet of Python coding shows some of buttons inside the main GUI window that uses grid frames to control each portion of the window in Tkinter library. Great tool to organize each command and their location inside the main or sub main windows. Another great method is to use 'lambda' function which is a great tool that combines multiple functions into one function and sends it as one command with one click of a button, which is an easy way remove any clutters while building your code.

```
Stop the vehicle at any point for emergncy or other things.
stop_vcle = tk.Button(control_frame, text="Stop Command", command=lambda:
stop_command(stop_vehicle),width=14, bg='#29465B',foreground='white')
stop_vcle.grid(row=6, column=1, padx=10,sticky='w')

#Continue the cycle button
run_vcle = tk.Button(control_frame, text="Continue Cycle", command=lambda:
continue_cycle(cnt_val), width=14,bg='#29465B',foreground='white')
run_vcle.grid(row=6, column=2, padx=10,sticky='w')

#Quit from the program button
quit_button = tk.Button(control_frame, text="Quit", command=quit_application,
bg='#29465B', foreground='white',width=14, anchor='center')
quit_button.grid(row=6, column=3, padx=20,sticky='w')

#this used to bind the window resized
window.bind('<Configure>', on_resize)

#tkinter loop
window.mainloop()
```

Python Application Recommendations

As mentioned previously, this Python application serves as a straightforward means of sending commands and controlling the driverless vehicle. However, there are several recommendations to enhance its functionality and performance:

- Communication Upgrade: While Bluetooth serial communication is suitable for short distances, it may become obsolete in complex warehouse environments. To address this limitation, consider replacing Bluetooth with more robust communication devices such as radio frequency transceivers, which can handle larger ranges and distances.
- Operation Logging: The application currently processes one message at a time, making it

suitable for individual instances. However, retrieving historical operation data or analyzing past behavior of the vehicle could be challenging. To overcome this, implement a logging database within the Python application to record and store vehicle operations. This will enable you to identify any irregularities or issues that occurred during operations.

- Multi-Operation Support: The application currently supports one cycle of operation at a time, which may not be sufficient for large-scale facilities. To address this limitation, consider introducing a customized instruction system that allows for multiple concurrent operations at various stations with different sets of colors. This will improve efficiency and scalability.
- Location Awareness: Currently, the controller lacks information about the vehicle's location. To enhance control and monitoring, incorporate a mapping system that provides real-time information on the vehicle's position. This can be achieved through vision-controlled sensors and speed correction mechanisms, ensuring better navigation and coordination within the facility.

These recommendations aim to improve the functionality and adaptability of the Python application for managing autonomous vehicles in complex warehouse environments.

Arduino Code

Our team will ensure that we elaborate on the most important aspects that allow our vehicle to operate. While we are also uploading our finalized code, our goal is to explain and discuss how the vehicle operates without delving into excessive technical details, thus enabling users to understand the roles various functions play throughout. Additionally, it is important to mention that some of the code in the LAFVIN smart car was used to assist in coding for the

Arduino microcontroller. This coding helped us learn and implement certain functions to achieve a smoother transition for what we added to the code, resulting in a fully functional program for the driverless vehicle (7_in_1_Multi_Function_Robot_Arm_Smart_Car, n.d.). Let's start by exploring the main sections.

Starting off, these are the different libraries that we defined which allowed us to reference code that was created by the manufacturers, allowing for easy implementation when referencing the hardware connected. This greatly simplified our coding experience and allowed us to correctly define the rules necessary to fulfill our functional requirements.

```
//Libraries to be used to control some of the sensor
#include <Adafruit_TCS34725.h>
#include <Wire.h>
#include <Servo.h>
#include <NewPing.h> // Include the NewPing library
#include <Adafruit_PWMServoDriver.h>
#include <SoftwareSerial.h>
```

This specific set of variables defined above allowed us to calibrate our color sensor, so that the colors were accurately being defined when our vehicle moved across the different colored paper. By having the machine understand the maximum and minimum red, green, and blue values of the light being reflected, we can specify a range that is acceptable and obtain values that are closer to the color being detected. To clarify, the code above is the range of values that the sensor itself can detect. However, we also specified specific ranges for each color of paper used such as red, orange, blue, and yellow.

```
//define the darkest color black will be minimum va
maximum value.
//this will calibrate all colors on these values.
int redMin = 28;
int redMax = 580;
int greenMin = 26;
int greenMax = 592;
int blueMin = 20;
int blueMax = 452;
```

This function simply allows our vehicle to flash LED lights when this function is called. We implement a half a second delay between when the LED light is enabled and disabled.

```
void alarmColorON() {
  pwm.setPWM(15,0,PWM_MAX);
  delay(500);
  pwm.setPWM(15,PWM_MAX,0);
  delay(500);

}
```

These functions allow our vehicle to make the appropriate movements when picking up/placing down a package from a colored station. Specifically, this is done by moving the appropriate joints based on the defined angle to seamlessly reach the object. Delays are incorporated to ensure the vehicle does not destroy platforms and to ensure the safety of workers. Each station has its own unique function because of the various package placements and unique movements. This applies to both functions listed above.

```
void pick_itemOrange() {
  move_claw(160);
  delay(1000);
  base_movement(162);
  delay(1000);
  move_arm(120);
  delay(1000);
  move_claw(20);
  delay(1000);
  move_arm(30);
  delay(1000);
  base_movement(83);

}
```

```
void drop_itemOrange() {
  base_movement(166);
  delay(1000);
  move_arm(115);
  delay(1000);
  move_claw(165);
  delay(1000);
  move_arm(30);
  delay(1000);
  base_movement(83);
  delay(1000);
  move_claw(60);
}
```

At any given time, our team determined that it is important to know when a package is being held. In our code, this can easily be checked by checking the degree of the claw on our vehicle. If the claw degree is below our specified range in degrees, we know the item is picked up (isItemPicked()). If the claw degree is

```
// ---- checking to see if the item is picked or not ---- //
bool isItemPicked() {
  // Check the claw degree to see if an item is picked
  if(current_claw_degrees <= 20){
    return true;
  } else{
    return false;
  }

}

//check for the item if is dropped
bool isItemDropped() {
  // Check the claw degree to see if an item is picked
  if(current_claw_degrees >= 50){
    return true;
  } else{
    return false;
  }

}
```

over the threshold, we know the item is dropped (isItemDropped()). From a coding perspective, we can return a true or false value and store it for future reference.

This function is used when we want our vehicle to check for our defined colors that were defined inside the program. This works by constraining the values obtained by the color sensor from a range of 0 – 255. This allows us to align the color values obtained from the sensor itself with the values that we setup for each color on an individual basis. If these values do align with a color that we previously defined, the function will return a true value and the appropriate operation will be performed.

```
// this is the function will check will bool of the color set to the reading of
the sensor the time of pickup or drop off
bool colorMatch() {
  // Read RGB color values from the sensor
  uint16_t red, green, blue, clear;
  tcs.getRawData(&red, &green, &blue, &clear);

  int redV = map(red,redMin, redMax, 0,255);
  int greenV = map(green, greenMin, greenMax, 0, 255);
  int blueV = map(blue,blueMin,blueMax,0,255);

  //constrain to values between 0-255 not to go over these values
  int redVal = constrain(redV,0,255);
  int greenVal=constrain(greenV, 0,255);
  int blueVal=constrain(blueV,0,255);


  // Check for color matches and update the flags
  isColorOrange = isOrange(redVal, greenVal, blueVal);
  isColorGreen = isGreen(redVal, greenVal, blueVal);
  isColorYellow = isYellow(redVal, greenVal, blueVal);
  isColorLightBlue = isLightBlue(redVal, greenVal, blueVal);
  isColorDarkBlue = isDarkBlue(redVal, greenVal, blueVal);
  isColorBlue = isBlue(redVal, greenVal, blueVal);
  isColorDarkOrange = isDarkOrange(redVal, greenVal, blueVal);
  isColorPurple = isPurple(redVal, greenVal, blueVal);
  isColorPink = isPink(redVal, greenVal, blueVal);
  isColorRed = isRed(redVal, greenVal, blueVal);


  // Return true if any color is detected, otherwise false
  return isColorOrange || isColorGreen || isColorYellow || isColorLightBlue ||
isColorDarkBlue || isColorDarkOrange || isColorPurple || isColorPink ||
isColorRed || isColorBlue;
}
```

One of the main functions is a snippet to show different types of operations of picking up from orange station and dropping of the object at blue station. There are multiple types of Boolean operation to tackle each condition and make sure that there are no validation or logical errors during the operation of the driverless vehicle.

```
//same as the function above with tackling different colors
void orange_blue_pick_drop(){
  orange_blue_PickDrop_flag = true;
  alarmColorOFF();

  bool pickUpPerformed = false;
  bool dropOffPerformed = false;
  bool isLineTracking = true;
  bool colorMatchStarted = true;

  unsigned long currentAvoidTimer = 0;
  unsigned long currentTime = 0;
  unsigned long continueTimer = 0;
  unsigned long pickUpTimer = 0; // Timer for pick-up delay
  unsigned int stopDistance = 15;
  unsigned int  stepCnt= 1;

  while(orange_blue_PickDrop_flag){
    currentCycle = 2;
    if(carState ==RUNNING){
      HC05.println("ORANGE - BLUE CYCLE!!");
      Left_Tra_Value = digitalRead(7);
      Center_Tra_Value = digitalRead(8);
      Right_Tra_Value = digitalRead(A1);
      unsigned int distance_cm = sonarItem.ping_cm();
      unsigned int distanceAvoid_cm = sonarAvoid.ping_cm();
```

This section of our code demonstrates a full lap/circle for when our autonomous vehicle picks up/drops off an item from the orange station. Starting with the first set of variables that are defined, our team ensured that the variables such as “pickUpPerformed”, “isLineTracking”, and other variables are set appropriately because we need to initialize our vehicle correctly by stating the default conditions. We also made sure to define our timers that were used throughout our program. Because we are only looking at the conditions inside of the first lap/circle, our team used a while loop to repeat any conditions that were inside of it. Specifically, inside this loop, we made sure to define the pins that are associated with our line-tracking sensors and defined the two situations/paths the vehicle can travel through once it approaches the dual-lane section. For example, the vehicle can travel straight when encountering the two paths if there are no obstacles; otherwise, the vehicle may take an alternative path by turning left (if the path is unobstructed and available). Progressing with the existing code, if the above conditions are not relevant to the vehicle’s current state (conditions not met), the vehicle will jump to a different set of instructions which looks to see if there is an object 10 centimeters away from the ultrasonic sensors. If there is an object within this distance, the vehicle will simply halt all operations and flash the alarming LED light until the object is removed. Once removed, the vehicle will proceed with its intended operation. These

```
void jobOne() {
  alarmColorOFF();
  jobOne_flag = true;
  jobCycleCounter = 1;

  bool pickUpPerformed = false; // Flag to track whether pick-up action has been performed
  bool dropOffPerformed = false;
  bool isLineTracking = true;
  bool colorMatchStarted = true;

  unsigned long currentAvoidTimer = 0;
  unsigned long currentTime = 0;
  unsigned long continueTimer = 0;
  unsigned long pickUpTimer = 0; // Timer for pick-up delay
  unsigned int stopDistance = 15;

  unsigned int  stepCnt= 1;
  while(jobOne_flag && jobCycleCounter == 1) {
    currentCycle = 11;
    if(carState == RUNNING){
      HC05.println("CYCLE 1: ORANGE - ORANGE STATIONS!!");
      Left_Tra_Value = digitalRead(7);
      Center_Tra_Value = digitalRead(8);
      Right_Tra_Value = digitalRead(A1);
      unsigned int distance_cm = sonarItem.ping_cm();
      unsigned int distanceAvoid_cm = sonarAvoid.ping_cm();

      if(distanceAvoid_cm <=40 && distanceAvoid_cm > 10){
        if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
          isLineTracking = false;
          Rotate_Left(75);
          delay(200);
          isLineTracking = true;
          Serial.println("if statement <= 40cm > 10cm");
        }
      }
      else if(distanceAvoid_cm > 40|| distanceAvoid_cm == 0){
        if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
          isLineTracking = false;
          Move_Forward(60);
          delay(100);
          isLineTracking = true;
          Serial.println("if statement >40cm");
        }
      }

      switch (stepCnt){
        case 1:
          if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
```

operations take priority and is checked first. Afterward, if a package is picked up and an obstructive object is detected in front of the vehicle, the same function as previously mentioned occurs and the object is held. Also, we set up a timer, so that other colors are ignored when an item is picked up temporarily. This process was coded with switch statements and depending on whether the vehicle was in the process of picking up, transferring the package, or dropping it off, we simply referenced the specific case number to ensure that our obstacle avoidance doesn't interfere with the original task given. This process of picking up a package is repeated 2 more times but with different colored tape. Lastly, when a singular package is picked up, dropped off, and reaches the red line (which marks the end of the current lap), our team considered this a lap/circle.

Arduino Code Recommendations

These recommendations are meant to improve the efficiency and robustness of our coding process, particularly during the early stages. Given more time, we can implement these suggestions to address potential issues without the need for further reconsideration.

- Code Optimization: Consider refactoring the code to reduce its size and memory footprint. Employ techniques such as code modularization, minimizing redundant functions, and using more efficient data structures to streamline the codebase.
- Efficient Algorithms: Explore and implement more efficient algorithms for color detection and data processing. Optimized algorithms can enhance the speed and reliability of communication between the Arduino controller and the Python control center application.
- Multi-Operation Handling: Develop a more versatile codebase capable of handling multiple operations concurrently. This can be achieved by creating a structured system

that manages various tasks simultaneously, allowing for improved efficiency and flexibility.

- Color Customization Classes: Introduce classes or configuration files that enable easy customization of color definitions for each station. This approach simplifies color management, making it effortless to adapt the system to different station requirements without modifying the core code.

These recommendations aim to enhance the Arduino code's efficiency, flexibility, and maintainability for the driverless vehicle's color sensing and station operations.

Socioeconomic and Environmental Effects

Our group believes that our project has a major effect on various industries, especially online retailers such as Amazon. This is because our vehicle assists with repetitive tasks that requires minimal human interaction. Our vehicle could potentially replace physical workers who move packages across a warehouse, for example. However, we believe some industries may choose to continue having workers complete these mundane tasks because of how critical/sensitive some items are and how important the task is. While our technology has greatly evolved over the years, it may be early to put these new machines to use in a full-production environment; however, other companies may have different ideologies. Some companies will benefit when it comes to paying for these machines because of their fixed price (opposed to paying a person hourly), but our solution may not supply the needs of all companies. Our product may also reduce congestion when moving items across an environment because of the size and how efficient these vehicles are. When a human is performing these tasks, the time can vary because of their work ethic and motivation; however, a vehicle like ours would move packages at a consistent rate. This would allow for increased productivity, encouraging businesses to

capitalize on the latest technologies. While machines may not be the optimal solution for all companies, especially because of their needs and focus, there is certainly a small but niche community that is looking for this type of technology to be developed for an indoor setting.

Test Case Scenarios:

Test Case #	**Specific Application**	**Description**	**Expected Output**	**Actual Output**	**Pass/Fail**
1	Autonomous Delivery Vehicle	Follow Black Line (Default Path)	The vehicle will follow the black line when colored tape is not detected.	The vehicle will follow the black line when colored tape is not detected.	**Pass**
2	Autonomous Delivery Vehicle	Pick-up Package (Orange Station)	The vehicle will stop at the first set of orange tape, pick-up the package, and proceed with its route.	The vehicle will stop at the first set of orange tape, pick-up the package, and proceed with its route.	**Pass**
3	Autonomous Delivery Vehicle	Drop off Package (Orange Station)	The vehicle will stop at the second set of orange tape, drop-off the package, and proceed with its route.	The vehicle will stop at the second set of orange tape, drop-off the package, and proceed with its route.	**Pass**
4	Autonomous Delivery Vehicle	Pick-up Package (Blue Station)	The vehicle will stop at the first set of blue tape, pick-up the package, and proceed with its route.	The vehicle will stop at the first set of blue tape, pick-up the package, and proceed with its route.	**Pass**
5	Autonomous Delivery Vehicle	Drop-off Package (Blue Station)	The vehicle will stop at the second set of blue tape, drop-off the package, and proceed with its route.	The vehicle will stop at the second set of blue tape, drop-off the package, and proceed with its route.	**Pass**
6	Autonomous Delivery Vehicle	Pick-up Package (Purple Station)	The vehicle will stop at the first set of purple tape, pick-up the package, and proceed with its route.	The vehicle will stop at the first set of purple tape, pick-up the package, and proceed with its route.	**Pass**

7	Autonomous Delivery Vehicle	Drop-off Package (Purple Station)	The vehicle will stop at the second set of purple tape, drop-off the package, and proceed with its route.	The vehicle will stop at the second set of purple tape, drop-off the package, and proceed with its route.	**Pass**
8	Autonomous Delivery Vehicle	Obstacle Avoidance	When the vehicle reaches the two paths available, if there is an object in the primary lane (straight), it will turn left and take the second path. If there is also an object in the second lane, all vehicle operations will halt until object is removed.	When the vehicle reaches the two paths available, if there is an object in the primary lane (straight), it will turn left and take the second path. If there is also an object in the second lane, all vehicle operations will halt until object is removed.	**Pass**
9	Autonomous Delivery Vehicle	Obstacle Detection	When an object/obstacle is present in the vehicle's path, all operations will halt, until the object is removed. Once removed, vehicle will proceed with its route.	When an object/obstacle is present in the vehicle's path, all operations will halt, until the object is removed. Once removed, vehicle will proceed with its route.	**Pass**
10	Autonomous Delivery Vehicle	Alarm Signal (LED Light)	When obstacle detection is occurring, flash the LED light, informing the appropriate team to investigate the issue.	When obstacle detection is occurring, flash the LED light, informing the appropriate team to investigate the issue.	**Pass**
11	Autonomous Delivery Vehicle	New Lap/Circle Detected	The vehicle will briefly stop at the red tape, indicating that a new lap/circle has started.	The vehicle will briefly stop at the red tape, indicating that a new lap/circle has started.	**Pass**

Final Results

After continuously working on this project, physically building the vehicle, and constructing the software, we were able to successfully meet our functional requirements that were set at the start of the project. After performing several tests with our packages and testing the pick-up functionality, we have not had any instances of objects being dropped since the code was finalized. We obtained the same result when objects are being picked up. However, our team noted that we need to be careful because of the lighting of the warehouse which could play a role with the color sensor, affecting the pickup location of an object. If the lighting was too bright, for example, it may change how the color red is defined, making our vehicle take no action if passed by the colored tape. While our testing was done in an environment where the lighting is consistent, we understood that this is a possibility and should account for it. Furthermore, our vehicle has successfully managed to follow the black line by default, assuming that there are no obstacle or colored tape being detected.

Even though we have encountered errors previously when following the black line with the alternative path, this has been rectified and found both paths are functioning correctly and found the vehicle will follow it by default. If the vehicle detects a color while following the default path, the vehicle will perform the appropriate action by picking up an item, dropping off an item, or marking the end of a lap. These operations are successful with no errors occurring. The vehicle moves seamlessly and transfers between tasks without issues.

Lastly, when there is an object/obstacle placed in front of the vehicle's path, the vehicle immediately halts and does not perform any actions until the hazard has been removed. The LED light attached to the back of the vehicle will also flash, indicating that assistance is needed. Once the obstacle is removed, the vehicle proceeds with its route and continues the delivery process.

Lessons Learned

There are numerous lessons to be obtained from this project, especially because of the challenges that our team had to face. While all team members agreed that our product was successful and was completed in a timely manner, there are always improvements to be made to allow for greater efficiency and better problem resolution. Let's start by reviewing the different problems that were encountered and how they were resolved. Afterward, we can discuss how we can learn from these mistakes and improve future projects.

Lesson 1

When originally purchasing our vehicle and viewing the hardware that made the vehicle function, it looked very promising and appeared that it would serve our purpose perfectly. However, after receiving the product, we noticed that there were no instructions provided that explained how to build the vehicle. After checking the manufacturer's website, we still did not find a clear guide that explained how to put the vehicle together. Instead, we found documents that elaborated how to connect to the Bluetooth receiver that was included with the vehicle. When opening the document, it contained images of the vehicle already built. We did; however, manage to find YouTube videos that allowed us to successfully build our vehicle. Before purchasing our vehicle, our group should have ensured that we understood the contents that were going to be included in the package along with the complexity of the build, especially because of the excessive time that was dedicated and the miniscule struggles that were encountered along the way Additionally, we did not expect certain pieces to flex easily and break as easily as they did, especially with the small amount of force that was applied to certain pieces around the base of the claw. Thankfully, we were able to create the necessary pieces using a 3D printer; however, there was a chance that we would've had to postpone our product, limiting the time our group

had to build a high-quality product. A major takeaway is to ensure that we set standards of what to expect when receiving the item so that when these standards aren't met, we have a backup plan ready that contains alternative vehicles to purchase.

Lesson 2

When starting this project, our team created a Ghantt chart to assist with our project scheduling and attempted to use this chart to lead us in the right direction and allow for a smooth transition when completing tasks. Some of our tasks included research and collect parts, hardware assembly, software development, and more. However, because we needed to learn how the hardware could be integrated into our project and were unsure of the amount of time that would be taken, it was difficult to list the specifics of what we planned to do specifically within the broad description we provided. While our group managed to successfully complete the project in a timely manner and managed to exceed our own expectations, we definitely would have attempted to narrow down the task description, giving us more confidence with where the project was headed. While this could have put our project at risk because of potential extended deadlines, our teamwork and cooperation allowed us to overcome this obstacle.

Overall, our team was proud of the work we have accomplished, especially because of the timeframe that we were provided with. While we did encounter some minor errors when developing the software, these were predicted and handled appropriately.

Conclusion

This project has been a journey for our team, especially because we envisioned a society where technology would be affecting our everyday lives in plenty of ways. The wide range of ideas for technological devices/equipment made choosing a project difficult but enjoyable because we had an opportunity to learn about the different industries technology can be applied to. When we finally decided that we wanted to create an autonomous vehicle, we understood that we would be taking on a challenge because of the complexity and challenges that the vehicle (and ourselves) would encounter.

By researching and analyzing the components that were available in the hardware industry and seeking the benefits and drawbacks of each, we were able to order the appropriate parts and learn how each could be configured to properly assist in fulfilling our functional requirements. This was an excellent idea to implement because even though our team was unsure of the amount of progress that would be completed in the limited timeframe, we could easily define an end-point without stressing about additional items that were not implemented.

However, our team was able to thrive in our environment and go beyond our original expectations, creating an advanced piece of useful technology. The effects are invaluable, especially to an organization that is searching for a reliable, efficient vehicle to transport packages. While the task may appear unnecessary for a human-being, machines were meant to perform these simple, repetitive tasks without the need for human interaction. It's incredible to see our vehicle perform many tasks and tests which have taken excessive amounts of time and preparation, especially with how inexpensive it is. While it is highly sophisticated, the costs associated do not need to reflect the abilities it performs. Furthermore, each function has been individually tested and passed to ensure the safety and comfortable use of the product.

Throughout the experience of making this creation possible, valuable lessons were learned that will affect future projects and will be incorporated to improve our growing technological world.

References

7_in_1_Multi_Function_Robot_Arm_Smart_Car. (n.d.). Dropbox. https://www.dropbox.com/sh/27lu63pure2m9rx/AABEsYMiRk77TF2pcMmqW4IKa/Arduino_Code/Lesson_9/7_in_1_Multi_Function_Robot_Arm_Smart_Car?dl=0&subfolder_nav_tracking=1

Amazon.com: LAFVIN Mechanical 4WD Robot Arm Smart Car Kit Robot STEM/Graphical Programming Robot Car Compatible with Arduino IDE with Tutorial : Toys & Games. (n.d.). https://www.amazon.com/LAFVIN-Mechanical-Graphical-Programming-Compatible/dp/B09NSJLLNQ/ref=sr_1_7?crid=1GPB83UKYDR4V&keywords=lafvin&qid=1697580109&sprefix=lafvin%2Caps%2C102&sr=8-7

ChhayLy Lim. (2020, February 2). *Control Arduino with HC-06 wireless from PC using Python* [Video]. YouTube. https://www.youtube.com/watch?v=29AoZ94MLbc

Engineers, L. M. (2023, May 10). How HC-SR04 ultrasonic sensor works & Interface it with Arduino. *Last Minute Engineers*. https://lastminuteengineers.com/arduino-sr04-ultrasonic-sensor-tutorial/

I2C tutorial. (n.d.). https://www.robot-electronics.co.uk/i2c-tutorial#:~:text=I2C%20Device%20Addressing&text=This%20means%20that%20you%20can,it%20or%20reading%20from%20it.

ISO 9001:2015 - What is the 9001:2015 Standard? | ASQ. (n.d.). https://asq.org/quality-resources/iso-9001

NeuralNine. (2023b, January 10). *Python threading explained in 8 minutes* [Video]. YouTube. https://www.youtube.com/watch?v=A_Z1lgZLSNc

Ping Ultrasonic Range Finder | Arduino Documentation. (n.d.). https://docs.arduino.cc/built-in-examples/sensors/Ping

Simhoni, S. (2023, July 19). GM EV and battery investment in Michigan. *Center for American Progress*. https://www.americanprogress.org/article/gm-ev-and-battery-investment-in-michigan/

Szczepaniak, A. (2023, May 15). *Leo Rover Blog - Raspberry Pi or Arduino – when to choose which?* https://www.leorover.tech/post/raspberry-pi-or-arduino-when-to-choose-which#:~:text=Key%20Differences&text=Arduino%20is%20an%20electronic%20board,monitoring%20weather%20and%20many%20others.

TAOS. (2012). *TCS3472 color light-to-digital converter with IR filter*. cdn-shop Adafruit. https://cdn-shop.adafruit.com/datasheets/TCS34725.pdf

Tech with Mike. (2019, December 4). *How to build an arduino smart car (LAFVIN)* [Video]. YouTube. https://www.youtube.com/watch?v=MF-FqZn5e5M

Vishay. (n.d.). *TCRT5000, TCRT5000L Optical Sensors | Vishay*. https://www.vishay.com/en/product/83760/

Zero Congestion with Self-driving Vehicles | General Motors. (n.d.). https://www.gm.com/commitments/path-to-autonomous

Appendices

Python Code

Blew is the completed program of the front-end Python application applied in Visual Studio Code platform to control the driverless vehicle using Bluetooth communication protocols.

```
#---  Senior Project - Fall 2023  ---
#---  Autonomous Delivery Vehicle  ---
#---  By: Laith Fakhruldin and Spencer Martinez  ---
#---  Dr. Ece Yaprak and prof. Moise Sunda  ---
#---  ET4999  ---
#---  10/28/2023  ---
# This program is designed for controlling miniature autonomous vehicle in a
warehouse environment
# using a Bluetooth connection to an Arduino device. It offers various commands
to control
# the vehicle's movements, pick-up, and drop-off actions. The GUI interface
allows users
# to select pick-up and drop-off stations, establish a Bluetooth connection, send
commands,
# and monitor the vehicle's status. Additionally, it provides options for
resetting, stopping,
# continuing, and quitting the application.

# Required libraries for this application include tkinter for GUI, serial for
serial
# communication with the Arduino, os for system-related operations, time for
timing delays,
# and tkinter.messagebox for displaying messages.

#these libraries are required to complete this application and connect with
Arduino program
import serial
import tkinter as tk
import os
import time
from tkinter import simpledialog
import tkinter.messagebox as messagebox
import threading

connected = False # bool to check the connection status
ser = None  # Initialize serial port as None
sending_color = None # initialize command character to be sent to Arduino
```

```
com_port = None # initialize COM port to be as COM1,COM2, etc.

stop_vehicle = 's' #command value to stop vehicle
reset_val = 'g' #command value to reset vehicle
sim_val = 'i'   # simulation command
cnt_val = 'n'   #continue cycle command

# method for the Bluetooth Connection
def is_bluetooth_connected():
    return ser is not None

#This method to retrieve data from the Arduino side while the vehicle is running.
def retrieveData():
    global ser
    try:
        if ser is not None and ser.is_open:
            #retrieve the data from the arduino side as readline from serial port
"ser"
            data = ser.readline().decode('utf-8')
            if data:
                data_label.config(text=f"Response: {data}")
    except serial.SerialException as e:
         data_label.config(text=f"Serial Error: {str(e)}")  # Return the error
message


#using thread here for data retrieval, so there are no conflicts will occur while
running windows.loop() of tkinter
def data_retrieve_thread():
    while True:
        retrieveData()


data_thread = threading.Thread(target=data_retrieve_thread)
data_thread.daemon = True #Daemonize the thread to exit when main loop of windows
exits
data_thread.start()


def get_com_port():

    # Small popup window to insert the COM port into text field
    com_window = tk.Toplevel(window)
    com_window.geometry('{}x{}'.format(260, 150))
    com_window.title("COM - PORT")
```

```
    com_port_label = tk.Label(com_window, text="Enter COM Port:", bg='#89A3B6')
    com_port_label.grid(row=4, column=1, padx=10, sticky='w')
    com_port_entry = tk.Entry(com_window, bg="white")
    com_port_entry.grid(row=4, column=2, padx=10, sticky='w')

    # Definition to add the connection of serial port to the Bluetooth in Arduino
side
    def ok_button_clicked():
        #using some global variables for connection
        global ser, connected, com_port

        if connected:
            btooth_label.config(text="Bluetooth is already connected.")
        else:
          com_port = com_port_entry.get()
          connected = True

        if not com_port:
            #popup  error message window if you didn't add the COM port
            messagebox.showerror("Error", "Please enter a COM port.")
            connected = False
        else:
            try:
                #This is the connection to the serial connection with Bluetooth
on the Arduino side
                ser = serial.Serial(com_port, 9600, timeout=1)
                btooth_label.config(text="Bluetooth Connected")
                com_window.destroy()  # Close the COM port input window
            except serial.SerialException as e:
                btooth_label.config(text=f"Bluetooth Connection Error: {str(e)}")

    ok_button = tk.Button(com_window, text="OK", command=ok_button_clicked)
    ok_button.grid(row=2, column=2, sticky='nsew')



# Function to disconnect from Bluetooth
def disconnect_bluetooth():
    global ser, connected
    if connected:
        ser.close()
        btooth_label.config(text="Bluetooth Disconnected")
        connected = False
    else:
```

```
        messagebox.showinfo("Info", "Bluetooth is not connected.")


#defention to send the multicolored option to Arduino
def send_command(sending_color):
    global ser
    if ser == None:
        messagebox.showerror("Error", "Bluetooth is not connected.")
        return
    else:
        try:
            command = f"{sending_color}"
            ser.write(command.encode())
            time.sleep(0.1)  # Adjust the delay as needed
            #data = retrieveData()
           # data_label.config(text=f"Response: {data}")
        except serial.SerialException as e:
            data_label.config(text=f"Serial Error: {str(e)}")

def sim_command(sending_color):
    global ser
    if ser==None:
        messagebox.showerror("Error", "Bluetooth is not connected.")
        return
    else:
      try:
        command = f"{sending_color}"
        ser.write(command.encode())
        time.sleep(0.1)  # Adjust the delay as needed
        #data = retrieveData()
       # data_label.config(text=f"Response: {data}")
      except serial.SerialException as e:
        data_label.config(text=f"Serial Error: {str(e)}")

# this will stop the vehicle at any moment in any case.
def stop_command(sending_color):
    global ser
    if ser==None:
        messagebox.showerror("Error", "Bluetooth is not connected.")
        return
    else:
        command = f"{sending_color}"
        ser.write(command.encode())
        #time.sleep(0.1)
        data = retrieveData()
```

```
        data_label.config(text=f"Response: {data}")
        if ser:
            try:
                command = f"{sending_color}"
                ser.write(command.encode())
                time.sleep(0.1)  # Adjust the delay as needed
                #data = retrieveData()
                #data_label.config(text=f"Response: {data}")
            except serial.SerialException as e:
                data_label.config(text=f"Serial Error: {str(e)}")
        #else:
        #    data_label.config(text="Serial port is not initialized.")

# this will make continue the cycle where it was left after clicking stop command
def continue_cycle(sending_color):
    global ser
    if ser==None:
        messagebox.showerror("Error", "Bluetooth is not connected.")
        return
    else:
        command = f"{sending_color}"
        ser.write(command.encode())
        #time.sleep(0.1)
        data = retrieveData()
        data_label.config(text=f"Response: {data}")
        if ser:
            try:
                command = f"{sending_color}"
                #This is an important piece of code when trying to send command
to the other side or Arduino is ser.write(....)
                ser.write(command.encode())
                time.sleep(0.1)  # Adjust the delay as needed
                #data = retrieveData()
                #data_label.config(text=f"Response: {data}")
            except serial.SerialException as e:
                data_label.config(text=f"Serial Error: {str(e)}")
        #else:
        #    data_label.config(text="Serial port is not initialized.")

# reset the values to default
def reset_command(sending_color):
    global ser
    if ser==None:
        messagebox.showerror("Error", "Bluetooth is not connected.")
        return
```

```
    else:
        data_label.config(text=f"")
        command = f"{sending_color}"
        ser.write(command.encode())
        #time.sleep(0.1)
        data = retrieveData()
        data_label.config(text=f"Response: {data}")
        if ser:
            try:
                command = f"{sending_color}"
                ser.write(command.encode())
                time.sleep(0.1)  # Adjust the delay as needed
                #data = retrieveData()
                #data_label.config(text=f"Response: {data}")
            except serial.SerialException as e:
                data_label.config(text=f"Serial Error: {str(e)}")
        #else:
        #    data_label.config(text="Serial port is not initialized.")

#this function will close the application exit from the program
def quit_application():
    global ser
    if ser is not None:
        if ser.is_open:
            ser.close()
    os._exit(0)


#resize the window scale
def on_resize(event):
    new_width = event.width
    top_frame.config(width=new_width)

#this function will help in determining what command to send when choosing
different color in radio buttons.
def get_pick_drop():
    #getting the values of each radio button
    pick_color = pickup_color_var.get()
    drop_color = dropoff_color_var.get()

    #this is just to check values of the radio buttons in console
    print("Pickup Color:", pick_color)
    print("Drop-off Color:", drop_color)

    # Initialize sending_color with a default value
```

```
    if pick_color == 'o' and drop_color == 'r':
        sending_color = 'o'
    elif pick_color == 'o' and drop_color == 'l':
        sending_color = 'b'
    elif pick_color == 'o' and drop_color == 'u':
        sending_color = 'p'
    elif pick_color == 'b' and drop_color == 'r':
        sending_color = 'l'
    elif pick_color == 'b' and drop_color == 'l':
        sending_color = 'e'
    elif pick_color == 'b' and drop_color == 'u':
        sending_color = 'm'
    elif pick_color == 'p' and drop_color == 'r':
        sending_color = 'c'
    elif pick_color == 'p' and drop_color == 'l':
        sending_color = 'd'
    elif pick_color == 'p' and drop_color == 'u':
        sending_color = 'a'
    print("SendingColor: ", sending_color)
    return sending_color

""" --- This part of the code will handle the window and any GUI commands in this
program --- """
# Create a tkinter window
window = tk.Tk()
window.title("Senior Project")
window.geometry('{}x{}'.format(940, 600))
window.resizable(True, True)

#start to retrieve data from Arduino side
retrieveData()

top_frame = tk.Frame(window, bg='#29465B', height=55, pady=5, padx=10)
top_frame.grid(row=0, columnspan=20, sticky='nesw')

copyright = tk.Label(top_frame, bg='#2E8B57', foreground='white', text='--- All
Rights Reserved - Copyright 2023 - Autonomous Delivery Vehicle --- ',
font='Helvetica 12', height=2, width=80)
copyright.grid(row=0, column=1, columnspan=20,  sticky='n')
window.grid_columnconfigure(0, weight=1)
top_frame.grid_columnconfigure(1,weight=1)

# adding control buttons in this frame
control_frame = tk.Frame(window, bg='#89A3B6', width=600, height=100, pady=5,
padx=100)
```

```
control_frame.grid(row=1, column=0, columnspan=20, rowspan=20, sticky='nesw')

# changing the grid rows and columns for row 1 that is control_frame
window.grid_rowconfigure(1, weight=1)
window.grid_columnconfigure(1, weight=1)
control_frame.grid_columnconfigure(1,weight=0)
control_frame.grid_rowconfigure(1,weight=0)


#setting up the variables that will be taken from radio buttons.
pickup_color_var = tk.StringVar()
dropoff_color_var = tk.StringVar()
pickup_color_var.set(None)
dropoff_color_var.set(None)

#spacer to add between rows and columns

spacer2 = tk.Label(control_frame, text="",bg='#89A3B6').grid(row=6, column=0,
columnspan=5, rowspan=50)
spacer3 = tk.Label(control_frame, text="",bg='#89A3B6').grid(row=6, column=0,
columnspan=5, rowspan=50)
spacer4 = tk.Label(control_frame, text="",bg='#89A3B6').grid(row=6, column=0,
columnspan=5, rowspan=50)
spacer5 = tk.Label(control_frame, text="",bg='#89A3B6').grid(row=6, column=0,
columnspan=5, rowspan=50)
spacer6 = tk.Label(control_frame, text="",bg='#89A3B6').grid(row=6, column=0,
columnspan=5, rowspan=50)


# Color selection buttons for pickup and drop-off
color_label_pickup = tk.Label(control_frame, text="Pickup Station:",
bg='#89A3B6')
color_label_pickup.grid(row=2, column=0, sticky='w', padx=10)

color_label_dropoff = tk.Label(control_frame, text="Drop-off Station:",
bg='#89A3B6')
color_label_dropoff.grid(row=3,column= 0, sticky='w',padx=10)

#These are radio buttons are organized in three set for each pickup or drop off
station
orange_pick = tk.Radiobutton(control_frame, text="Orange",
variable=pickup_color_var, bg="orange",value="o",
highlightbackground="green").grid(row=2, column=1, sticky='w')
```

```
blue_pick = tk.Radiobutton(control_frame, text="Blue", variable=pickup_color_var,
bg="lightblue",value="b", highlightbackground="green").grid(row=2, column=2,
sticky='w')
purple_pick = tk.Radiobutton(control_frame, text="Purple",
variable=pickup_color_var, bg="#D8A7D8",value="p",
highlightbackground="green").grid(row=2, column=3, sticky='w')

orange_drop = tk.Radiobutton(control_frame, text="Orange",
variable=dropoff_color_var, bg="orange",value="r",
highlightbackground="green").grid(row=3, column=1, sticky='w')
blue_drop = tk.Radiobutton(control_frame, text="Blue",
variable=dropoff_color_var, bg="lightblue",value="l",
highlightbackground="green").grid(row=3, column=2, sticky='w')
pruple_drop = tk.Radiobutton(control_frame, text="Purple",
variable=dropoff_color_var, bg="#D8A7D8",value="u",
highlightbackground="green").grid(row=3, column=3, sticky='w')

# Create a "Connect" button
connect_button = tk.Button(control_frame, text="Connect Bluetooth",
width=17,command=get_com_port, bg='#29465B', foreground='white')
connect_button.grid(row=2, column=4, padx=10, sticky='w')

# Create a "Disconnect" button
disconnect_button = tk.Button(control_frame, text="Disconnect
Bluetooth",width=17, command=disconnect_bluetooth, bg='#29465B',
foreground='white')
disconnect_button.grid(row=3, column=4, padx=10,sticky='sw')

#This button for the multicolor command
button_send = tk.Button(control_frame, text="Send Command", command=lambda:
send_command(get_pick_drop()),width=14, bg='#29465B',foreground='white')
button_send.grid(row=4, column=0, padx=10,pady=12,sticky='w')

#Simulation button will activate three cycle command in Arduino
button_sim = tk.Button(control_frame, text="Simulation Cycles", command=lambda:
sim_command(sim_val),width=14, bg='#29465B',foreground='white')
button_sim.grid(row=4, column=1, padx=10,pady=12,sticky='w')

# Label for Bluetooth
btooth_status = tk.Label(control_frame, text="Bluetooth Status: ", bg='#89A3B6')
btooth_status.grid(row=4, column=4, columnspan=4, padx=5, sticky='w')

#This label to show the status of the Bluetooth it is connected
btooth_label = tk.Label(control_frame, text="", bg='#89A3B6')
btooth_label.grid(row=4, column=5, columnspan=4, padx=5, sticky='e')
```

```
# Adding a label for displaying the response
data_status = tk.Label(control_frame, text="Data Status: ", bg='#89A3B6')
data_status.grid(row=5, column=0, columnspan=1, padx=5,pady=10, sticky='nw')

#this is a data label will be used to receive any data from Arduino
data_label = tk.Label(control_frame, text="", bg='#89A3B6')
data_label.grid(row=5, column=1, columnspan=4, padx=5,pady=10, sticky='nw')

#Reset the vehicle
reset_button = tk.Button(control_frame, text="Reset Command", command=lambda:
reset_command(reset_val),width=14, bg='#29465B',foreground='white')
reset_button.grid(row=6, column=0, padx=10,sticky='w')

#Stop the vehicle at any point for emergncy or other things.
stop_vcle = tk.Button(control_frame, text="Stop Command", command=lambda:
stop_command(stop_vehicle),width=14, bg='#29465B',foreground='white')
stop_vcle.grid(row=6, column=1, padx=10,sticky='w')

#Continue the cycle button
run_vcle = tk.Button(control_frame, text="Continue Cycle", command=lambda:
continue_cycle(cnt_val), width=14,bg='#29465B',foreground='white')
run_vcle.grid(row=6, column=2, padx=10,sticky='w')

#Quit from the program button
quit_button = tk.Button(control_frame, text="Quit", command=quit_application,
bg='#29465B', foreground='white',width=14, anchor='center')
quit_button.grid(row=6, column=3, padx=20,sticky='w')

#this used to bind the window resized
window.bind('<Configure>', on_resize)


#tkinter loop
window.mainloop()
```

Arduino Code

This is the completed Arduino programming code where it is embedded into the Arduino microcontroller to the driverless vehicle.

```
/*
//---  Senior Project - Fall 2023  ---
//---  Autonomous Delivery Vehicle  ---
//---  By: Laith Fakhruldin and Spencer Martinez  ---
//---  Dr. Ece Yaprak and prof. Moise Sunda  ---
//---  ET4999  ---
//---  10/28/2023  ---

  This Arduino program is designed for controlling an autonomous miniature delivery vehicle.
  The vehicle is tasked with picking up and dropping off items at specific locations determined
  by the color of an RGB color sensor. It communicates with a Python application via Bluetooth
  to perform three-cycle simulations and receive commands for picking and dropping off items.

  The vehicle operates on a fixed tracking lane and will stop once a full cycle is complete,
  awaiting the next command from the Python program. Various functions and sensors are used
  to achieve precise movements and item handling, including the control of servos, motors,
  and ultrasonic sensors.

*/

//Libraries to be used to control some of the sensor
#include <Adafruit_TCS34725.h>
#include <Wire.h>
#include <Servo.h>
#include <NewPing.h> // Include the NewPing library
#include <Adafruit_PWMServoDriver.h>
#include <SoftwareSerial.h>


SoftwareSerial HC05(10,11); //(TX, RX) pins for the bluetooth communication

Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver(0x40);// address of the MCP23017 servo motor driver

//some global variables
#define SERVO1MIN 110
#define SERVO1MAX 540
#define SERVO2MIN 135
#define SERVO2MAX 500
#define SERVO3MIN 75
#define SERVO3MAX 535
#define SERVO_FREQ 50
#define LIGHT_OFF 1024
#define PWM_MAX 4096

// Define the pins used for motor control
const int DIRA_PIN = 2;
const int PWMA_PIN = 5;
const int DIRB_PIN = 4;
const int PWMB_PIN = 6;

//what channel each servo at start with 0 up to 15 "16 channels"
uint8_t servonum = 0;
```

```
//---- Define the TCS34725 sensor object ----//
Adafruit_TCS34725 tcs = Adafruit_TCS34725(TCS34725_INTEGRATIONTIME_50MS, TCS34725_GAIN_4X);

//define the darkest color black will be minimum value and whitest color white maximum value.
//this will calibrate all colors on these values.
int redMin = 28;
int redMax = 580;
int greenMin = 26;
int greenMax = 592;
int blueMin = 20;
int blueMax = 452;


// ---- defining the ultrasound sensorS for picking and dropping items and avoidance ---- //
#define TRIGGER_PIN A2
#define ECHO_PIN A3
#define TRIGGER_AVOID_PIN 12
#define ECHO_AVOID_PIN 13
#define MAX_DISTANCE 1000// Define a maximum distance value
NewPing sonarItem(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);
NewPing sonarAvoid(TRIGGER_AVOID_PIN, ECHO_AVOID_PIN, MAX_DISTANCE);


// ---- set global inputs and outputs for pins and functions ---- //

bool forward_flag;
bool backward_flag;
bool left_flag;
bool right_flag;
bool jobOne_flag;
bool jobTwo_flag;
bool jobThree_flag;
bool stop_flag;
bool alarm_flag;
bool isColorOrange;
bool isColorGreen;
bool isColorYellow;
bool isColorLightBlue;
bool isColorBlue;
bool isColorDarkBlue;
bool isColorDarkOrange;
bool isColorPurple;
bool isColorPink;
bool isColorRed;
bool isLineTracking;
bool shouldReset = false;
bool orange_orange_PickDrop_flag = false;
bool orange_blue_PickDrop_flag = false;
bool orange_purple_PickDrop_flag = false;
bool blue_blue_PickDrop_flag = false;
bool blue_orange_PickDrop_flag = false;
bool blue_purple_PickDrop_flag = false;
bool purple_purple_PickDrop_flag = false;
bool purple_orange_PickDrop_flag = false;
bool purple_blue_PickDrop_flag = false;
bool orangeDropOff_flag = false;
bool pickUpPerformed = false;
```

```
bool resumeCycle = false;

unsigned long pickupStartTime = 0;
const unsigned long pickupDelay = 2000;

static unsigned int jobCycleCounter;
int base_degrees;
int arm_degrees;
int claw_degrees;
int current_claw_degrees;
int colorPassCnt;
int Left_Tra_Value;
int Center_Tra_Value;
int Right_Tra_Value;
int Black_Line;
int distance;
char command;
int currentCycle = 0;

//struct is used to control the stopping moving of vehicle
enum CarState {
  RUNNING,
  STOPPED
};

CarState carState = RUNNING;


void alarmColorON() {
  pwm.setPWM(15,0,PWM_MAX);
  delay(500);
  pwm.setPWM(15,PWM_MAX,0);
  delay(500);
  /*for (int i=0;i<4096;i++) {
      pwm.setPWM(15, 0, i);
    }

    for (int i=4096;i>0;i--) {
      pwm.setPWM(15, i, 0);
    }
  */
}

void alarmColorOFF() {
  pwm.setPWM(15, PWM_MAX, 0); //PWM_MA = 4096,
}

// --- pickup and drop off item function  for each distinct color----//
void pick_itemOrange() {
  move_claw(160);
  delay(1000);
  base_movement(162);
  delay(1000);
  move_arm(120);
  delay(1000);
  move_claw(20);
  delay(1000);
  move_arm(30);
```

```
  delay(1000);
  base_movement(83);

}
void drop_itemOrange() {
  base_movement(166);
  delay(1000);
  move_arm(115);
  delay(1000);
  move_claw(165);
  delay(1000);
  move_arm(30);
  delay(1000);
  base_movement(83);
  delay(1000);
  move_claw(60);
}

void pick_itemBlue() {
  move_claw(160);
  delay(1000);
  base_movement(163);
  delay(1000);
  move_arm(175);
  delay(1000);
  move_claw(18);
  delay(1000);
  move_arm(30);
  delay(1000);
  base_movement(83);

}
void drop_itemBlue() {
  base_movement(163);
  delay(1000);
  move_arm(139);
  delay(1000);
  move_claw(165);
  delay(1000);
  move_arm(30);
  delay(1000);
  base_movement(83);
  delay(1000);
  move_claw(60);
}

void pick_itemPurple() {
  move_claw(165);
  delay(1000);
  base_movement(164);
  delay(1000);
  move_arm(123);
  delay(1000);
  move_claw(18);
  delay(1000);
  move_arm(30);
  delay(1000);
  base_movement(83);
```

```
}
void drop_itemPurple() {
  base_movement(164);
  delay(1000);
  move_arm(123);
  delay(1000);
  move_claw(165);
  delay(1000);
  move_arm(30);
  delay(1000);
  base_movement(83);
  delay(1000);
  move_claw(60);
}


// ---- checking to see if the item is picked or not ---- //
bool isItemPicked() {
  // Check the claw degree to see if an item is picked
  if(current_claw_degrees <= 20){
    return true;
  } else{
    return false;
  }

}

//check for the item if is dropped
bool isItemDropped() {
  // Check the claw degree to see if an item is picked
  if(current_claw_degrees >= 50){
    return true;
  } else{
    return false;
  }

}


//----- set the colors inputs from RGB color sensor ------//

// these settings are to initialize the colors that is picked from reading each color of RGB sensor --- //
bool isRed(unsigned int red, unsigned int green, unsigned int blue) {
  int redMinRed = 115;
  int redMaxRed = 185;
  int greenMinRed = 30;
  int greenMaxRed = 69;
  int blueMinRed = 40;
  int blueMaxRed = 90;
  // Check if the RGB values fall within the specified red range
  bool isColorInRangeResult = (red >= redMinRed && red <= redMaxRed && green >=greenMinRed && green <= greenMaxRed
&& blue >= blueMinRed && blue <= blueMaxRed);
  return isColorInRangeResult;
}

bool isBlue(unsigned int red, unsigned int green, unsigned int blue) {
  //set the threshold for each RGB of the sensor
```

```
  int redMinRed = 36;
  int redMaxRed = 74;
  int greenMinRed = 78;
  int greenMaxRed = 111;
  int blueMinRed = 130;
  int blueMaxRed = 203;
  bool isColorInRangeResult = (red >= redMinRed && red <= redMaxRed && green >=greenMinRed && green <= greenMaxRed
&& blue >= blueMinRed && blue <= blueMaxRed);
  return isColorInRangeResult;
}
bool isOrange(unsigned int red, unsigned int green, unsigned int blue) {
  int redMin = 125;
  int redMax = 255;
  int greenMin = 71;
  int greenMax = 145;
  int blueMin = 55;
  int blueMax = 95;
  bool isColorInRangeResult = (red >= redMin && red <= redMax && green >=greenMin && green <= greenMax && blue >=
blueMin && blue <= blueMax);
  return isColorInRangeResult;
}

bool isPurple(unsigned int red, unsigned int green, unsigned int blue) {
  int redMin = 115;
  int redMax = 180;
  int greenMin = 72;
  int greenMax = 90;
  int blueMin = 102;
  int blueMax = 140;
  bool isColorInRangeResult = (red >= redMin && red <= redMax && green >=greenMin && green <= greenMax && blue >=
blueMin && blue <= blueMax);
  return isColorInRangeResult;
}
bool isDarkOrange(unsigned int red, unsigned int green, unsigned int blue) {
  int redMin = 125;
  int redMax = 155;
  int greenMin = 57;
  int greenMax = 72;
  int blueMin = 50;
  int blueMax = 80;
  bool isColorInRangeResult = (red >= redMin && red <= redMax && green >=greenMin && green <= greenMax && blue >=
blueMin && blue <= blueMax);
  return isColorInRangeResult;
}

bool isDarkBlue(unsigned int red, unsigned int green, unsigned int blue) {
  int redMin = 25;
  int redMax = 65;
  int greenMin = 45;
  int greenMax = 65;
  int blueMin = 95;
  int blueMax = 145;
  bool isColorInRangeResult = (red >= redMin && red <= redMax && green >=greenMin && green <= greenMax && blue >=
blueMin && blue <= blueMax);
  return isColorInRangeResult;
}

bool isLightBlue(unsigned int red, unsigned int green, unsigned int blue) {
```

```
  int redMin = 115;
  int redMax = 170;
  int greenMin = 125;
  int greenMax = 185;
  int blueMin = 140;
  int blueMax = 195;
  bool isColorInRangeResult = (red >= redMin && red <= redMax && green >=greenMin && green <= greenMax && blue >=
blueMin && blue <= blueMax);
  return isColorInRangeResult;
}

bool isGreen(unsigned int red, unsigned int green, unsigned int blue) {
  int redMin = 74;
  int redMax = 100;
  int greenMin = 105;
  int greenMax = 185;
  int blueMin = 110;
  int blueMax = 125;
  bool isColorInRangeResult = (red >= redMin && red <= redMax && green >=greenMin && green <= greenMax && blue >=
blueMin && blue <= blueMax);
  return isColorInRangeResult;
}

bool isYellow(unsigned int red, unsigned int green, unsigned int blue) {
  int redMin = 140;
  int redMax = 255;
  int greenMin = 140;
  int greenMax = 255;
  int blueMin = 90;
  int blueMax = 125;
  bool isColorInRangeResult = (red >= redMin && red <= redMax && green >=greenMin && green <= greenMax && blue >=
blueMin && blue <= blueMax);
  return isColorInRangeResult;
}

bool isPink(unsigned int red, unsigned int green, unsigned int blue) {
  int redMin = 145;
  int redMax = 255;
  int greenMin = 118;
  int greenMax = 205;
  int blueMin = 130;
  int blueMax = 195;
  bool isColorInRangeResult = (red >= redMin && red <= redMax && green >=greenMin && green <= greenMax && blue >=
blueMin && blue <= blueMax);
  return isColorInRangeResult;
}

//this is the color of the foam board set as the road, may be needed it in the future
bool isWhite(unsigned int red, unsigned int green, unsigned int blue) {
  int redMin = 430;
  int redMax = 480;
  int greenMin = 450;
  int greenMax = 550;
  int blueMin = 450;
  int blueMax = 550;
  bool isColorInRangeResult = (red >= redMin && red <= redMax && green >=greenMin && green <= greenMax && blue >=
blueMin && blue <= blueMax);
  return isColorInRangeResult;
```

```
}

// this is the function will check will bool of the color set to the reading of the sensor the time of pickup or drop off
bool colorMatch() {
  // Read RGB color values from the sensor
  uint16_t red, green, blue, clear;
  tcs.getRawData(&red, &green, &blue, &clear);

  int redV = map(red,redMin, redMax, 0,255);
  int greenV = map(green, greenMin, greenMax, 0, 255);
  int blueV = map(blue,blueMin,blueMax,0,255);

  //constrain to values between 0-255 not to go over these values
  int redVal = constrain(redV,0,255);
  int greenVal=constrain(greenV, 0,255);
  int blueVal=constrain(blueV,0,255);
  //Serial.print("Red: "); Serial.print(redVal);
  //Serial.print(" - Green: "); Serial.print(greenVal);
  //Serial.print(" - Blue: "); Serial.print(blueVal);
  //Serial.println();
  //delay(1000);

  // Check for color matches and update the flags
  isColorOrange = isOrange(redVal, greenVal, blueVal);
  isColorGreen = isGreen(redVal, greenVal, blueVal);
  isColorYellow = isYellow(redVal, greenVal, blueVal);
  isColorLightBlue = isLightBlue(redVal, greenVal, blueVal);
  isColorDarkBlue = isDarkBlue(redVal, greenVal, blueVal);
  isColorBlue = isBlue(redVal, greenVal, blueVal);
  isColorDarkOrange = isDarkOrange(redVal, greenVal, blueVal);
  isColorPurple = isPurple(redVal, greenVal, blueVal);
  isColorPink = isPink(redVal, greenVal, blueVal);
  isColorRed = isRed(redVal, greenVal, blueVal);

  //we use these printout to check the values of the RGB color sensor
  /*Serial.print("Red: ");Serial.print(isColorRed);
  Serial.print(" - isOrange: ");Serial.print(isColorOrange);
  Serial.print(" - isGreen: ");Serial.print(isColorGreen);
  Serial.print(" - isYellow: ");Serial.print(isColorYellow);
  Serial.print(" - isDarkBlue: ");Serial.print(isColorDarkBlue);
  Serial.print(" - isPurple: ");Serial.print(isColorPurple);
  Serial.print(" - isPink: ");Serial.print(isColorPink);
  Serial.print(" - isLightBlue: ");Serial.print(isColorLightBlue);
  Serial.print(" - isDarkDarOrange: ");Serial.print(isColorDarkOrange);
  Serial.print(" - isPink: ");Serial.print(isColorPink);
  Serial.print(" - isBlue: ");Serial.print(isColorBlue);
  Serial.println();
  delay(1000);*/

  // Return true if any color is detected, otherwise false
  return isColorOrange || isColorGreen || isColorYellow || isColorLightBlue || isColorDarkBlue || isColorDarkOrange ||
isColorPurple || isColorPink || isColorRed || isColorBlue;
}

//These functions will determine received command from the Python with type of coloring of pickup and drop off
void orange_orange_pick_drop(){
  //activate the while loop for this function
  orange_orange_PickDrop_flag = true;
```

```
//set the alarm off
alarmColorOFF();
//different types of variables used for creating operation driving vehicle run smoothly
bool pickUpPerformed = false;
bool dropOffPerformed = false;
bool isLineTracking = true;
bool colorMatchStarted = true;

unsigned long currentAvoidTimer = 0;
unsigned long currentTime = 0;
unsigned long continueTimer = 0;
unsigned long pickUpTimer = 0; // Timer for pick-up delay
unsigned int stopDistance = 15;
unsigned int  stepCnt= 1;
//loop through to pick the item from orange station and drops it at orange station
while(orange_orange_PickDrop_flag){
  if(carState ==RUNNING){
    //used to get command 'n' for Coninue Cycle from Python run cycle again for this function only
    currentCycle = 1;
    //send a status report to Python program
    HC05.println("ORANGE - ORANGE CYCLE!!");
    //reading the tracking, ultrasonic sensors to see their conditions.
    Left_Tra_Value = digitalRead(7);
    Center_Tra_Value = digitalRead(8);
    Right_Tra_Value = digitalRead(A1);
    unsigned int distance_cm = sonarItem.ping_cm();
    unsigned int distanceAvoid_cm = sonarAvoid.ping_cm();
    //using front ultrasonic sensor to skip to a different route if object on the main route
    if(distanceAvoid_cm <=40 && distanceAvoid_cm > 10){
      if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
        isLineTracking = false;
        Rotate_Left(85);
        delay(200);
        isLineTracking = true;
        Serial.println("if statement <= 40cm > 10cm");
      }
    }
    //this let the vehicle to move forward on the main route if no obsetruction on the route.
    else if(distanceAvoid_cm > 40|| distanceAvoid_cm == 0){
      if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
        isLineTracking = false;
        Move_Forward(60);
        delay(100);
        isLineTracking = true;
        Serial.println("if statement >40cm");
      }
    }
    //added switch statement to add a timer between pickup and drop off stations do to conflict with same color
    //will be dropped of on the same station of pickup
    switch (stepCnt){
      //pickup station in the orange station.
      case 1:
        if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
          Stop();
          alarmColorON();
          isLineTracking = false;
          colorMatchStarted = false;
        }
```

```
  else{
    alarmColorOFF();
    colorMatchStarted = true;
    isLineTracking = true;
    if(isColorOrange && (distance_cm <= stopDistance && distance_cm > 0) && current_claw_degrees >=50) {
      isLineTracking = false;
      colorMatchStarted = false; // Reset the flag
      if(isItemDropped()){
        Stop();
        delay(200);
        pick_itemOrange();
        pickUpPerformed = true;
        stepCnt = 2;
      }
    }
    else if(isItemPicked()){
      stepCnt= 2;
    }
  }
  break;
//adding timer case condition
 case 2:
  if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
    Stop();
    alarmColorON();
    isLineTracking = false;
    colorMatchStarted = false;
  }
  else{
    alarmColorOFF();
    isLineTracking = true;
    colorMatchStarted = true;
    isLineTracking = true;
    if (isItemPicked()) {
      if(pickUpTimer == 0){
        pickUpTimer = millis();
      }
      if (millis() - pickUpTimer >= 3000) {
        pickUpPerformed = true;
        colorMatchStarted = true;
        pickUpTimer = 0;
        stepCnt = 3;
      }
    }
  }
  break;
 case 3:
  if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
    Stop();
    alarmColorON();
    isLineTracking = false;
    colorMatchStarted = false;
  }
  else{
    alarmColorOFF();
    colorMatchStarted = true;
    isLineTracking = true;
    if(pickUpPerformed && isColorOrange && current_claw_degrees <= 20 ){
```

```
        isLineTracking = false;
        colorMatchStarted = false; // Reset the flag
        if(isItemPicked()){
          Stop();
          delay(200);
          drop_itemOrange();
          dropOffPerformed = true;
          isLineTracking = true;
          colorMatchStarted = true;
          stepCnt = 4;
        }
      }
    }
    break;
  case 4:
    //this will halt the vehicle if any object reached within 10cm and will move when the object is removed
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
    }
    break;

}
//this will end the cycle of the picking and dropping off object condition
if(isColorRed){
  delay(150);
  Stop();
  delay(800);
  orange_orange_PickDrop_flag = false;
  stepCnt = 1;
  currentCycle =10;
  shouldReset=true;
  break;
}
//make the tracking condition true when needed
if(isLineTracking){
  black_line();
}
//make the color match condition true when required
if(colorMatchStarted){
  colorMatch();
}
//this condition will activate the stop command came from Python program side with 's'
if( HC05.available()>0){
  command = HC05.read();
  if(command == 's'){
    carState = STOPPED;
    Stop();
    HC05.println("Vehicle has been STOPPED!!");
  }
}
```

```
    }
    //this will continue the cycle until the red color is reached to end the cycle
    else if(carState == STOPPED){
      if(HC05.available()>0){
        command = HC05.read();
        if(command == 'n'){
          stepCnt = 1;
          currentCycle = 1;
          HC05.println("ORANGE - ORANGE CYCLE!!");
          carState = RUNNING;
        }
      }
    }
  }
}


//same as the function above with tackling different colors
void orange_blue_pick_drop(){
  orange_blue_PickDrop_flag = true;
  alarmColorOFF();

  bool pickUpPerformed = false;
  bool dropOffPerformed = false;
  bool isLineTracking = true;
  bool colorMatchStarted = true;

  unsigned long currentAvoidTimer = 0;
  unsigned long currentTime = 0;
  unsigned long continueTimer = 0;
  unsigned long pickUpTimer = 0; // Timer for pick-up delay
  unsigned int stopDistance = 15;
  unsigned int  stepCnt= 1;

  while(orange_blue_PickDrop_flag){
    currentCycle = 2;
    if(carState ==RUNNING){
      HC05.println("ORANGE - BLUE CYCLE!!");
      Left_Tra_Value = digitalRead(7);
      Center_Tra_Value = digitalRead(8);
      Right_Tra_Value = digitalRead(A1);
      unsigned int distance_cm = sonarItem.ping_cm();
      unsigned int distanceAvoid_cm = sonarAvoid.ping_cm();

      if(distanceAvoid_cm <=40 && distanceAvoid_cm > 10){
        if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
          isLineTracking = false;
          Rotate_Left(85);
          delay(200);
          isLineTracking = true;
          Serial.println("if statement <= 40cm > 10cm");
        }
      }
      else if(distanceAvoid_cm > 40|| distanceAvoid_cm == 0){
        if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
          isLineTracking = false;
          Move_Forward(60);
          delay(100);
```

```
    isLineTracking = true;
  }
}

switch (stepCnt){
  case 1:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
      if(isColorOrange && (distance_cm <= stopDistance && distance_cm > 0) && current_claw_degrees >=50) {
        isLineTracking = false;
        colorMatchStarted = false; // Reset the flag
        if(isItemDropped()){
          Stop();
          delay(200);
          pick_itemOrange();
          pickUpPerformed = true;
          stepCnt = 2;
        }
      }
      else if(current_claw_degrees <= 20){
        stepCnt= 2;
      }
    }
    break;
  case 2:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      isLineTracking = true;
      colorMatchStarted = true;
      isLineTracking = true;
      if (isItemPicked()){
        if(pickUpTimer == 0){
          pickUpTimer = millis();
        }
        if (millis() - pickUpTimer >= 12000) {
          pickUpPerformed = true;
          colorMatchStarted = true;
          pickUpTimer = 0;
          stepCnt = 3;
        }
      }
    }
    break;
  case 3:
```

```
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
      if(pickUpPerformed && isColorBlue  && current_claw_degrees <= 20){
        isLineTracking = false;
        colorMatchStarted = false; // Reset the flag
        if(isItemPicked()){
          Stop();
          delay(200);
          drop_itemBlue();
          dropOffPerformed = true;
          isLineTracking = true;
          colorMatchStarted = true;
          stepCnt = 4;
        }
      }
    }
    break;
  case 4:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
    }
    break;
}

if(isColorRed){
  delay(150);
  //pickUpPerformed = true;
  //dropOffPerformed = true;
  Stop();
  delay(800);
  orange_blue_PickDrop_flag = false;
  stepCnt = 1;
  currentCycle =10;
  shouldReset=true;
  break;
}
if(isLineTracking){
  black_line();
}

if(colorMatchStarted){
  colorMatch();
```

```
      }
      if( HC05.available()>0){
        command = HC05.read();
        if(command == 's'){
          carState = STOPPED;
          Stop();
          HC05.println("Vehicle has been STOPPED!!");
        }
      }
    }
    else if(carState == STOPPED){
      if(HC05.available()>0){
        command = HC05.read();
        if(command == 'n'){
          currentCycle = 2;
          HC05.println("ORANGE - BLUE CYCLE!!");
          carState = RUNNING;
        }
      }
    }
  }
}

//another function with different color
void orange_purple_pick_drop(){
  orange_purple_PickDrop_flag = true;
  alarmColorOFF();

  bool pickUpPerformed = false;
  bool dropOffPerformed = false;
  bool isLineTracking = true;
  bool colorMatchStarted = true;

  unsigned long currentAvoidTimer = 0;
  unsigned long currentTime = 0;
  unsigned long continueTimer = 0;
  unsigned long pickUpTimer = 0; // Timer for pick-up delay
  unsigned int stopDistance = 15;
  unsigned int  stepCnt= 1;

  while(orange_purple_PickDrop_flag){
    currentCycle = 3;
    if(carState == RUNNING){
      HC05.println("ORANGE - PURPLE CYCLE!!");
      Left_Tra_Value = digitalRead(7);
      Center_Tra_Value = digitalRead(8);
      Right_Tra_Value = digitalRead(A1);
      unsigned int distance_cm = sonarItem.ping_cm();
      unsigned int distanceAvoid_cm = sonarAvoid.ping_cm();

      if(distanceAvoid_cm <=40 && distanceAvoid_cm > 10){
        if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
          isLineTracking = false;
          Rotate_Left(85);
          delay(200);
          isLineTracking = true;
          Serial.println("if statement <= 40cm > 10cm");
        }
```

```
}
else if(distanceAvoid_cm > 40|| distanceAvoid_cm == 0){
  if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
    isLineTracking = false;
    Move_Forward(60);
    delay(100);
    isLineTracking = true;
    Serial.println("if statement >40cm");
  }
}

switch (stepCnt){
  case 1:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      Serial.print("stepCnt inside switch case 1: "); Serial.println(stepCnt);
      colorMatchStarted = true;
      isLineTracking = true;
      if(isColorOrange && (distance_cm <= stopDistance && distance_cm > 0) && current_claw_degrees >=50) {
        isLineTracking = false;
        colorMatchStarted = false; // Reset the flag
        if(isItemDropped()){
          Stop();
          delay(200);
          pick_itemOrange();
          pickUpPerformed = true;
          stepCnt = 2;
        }
      }
      else if(current_claw_degrees <= 20){
        stepCnt= 2;
      }
    }
    break;
  case 2:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
      stepCnt = 2;
      if(pickUpTimer == 0){
        pickUpTimer = millis();
      }
    }
    else{
      alarmColorOFF();
      isLineTracking = true;
      colorMatchStarted = true;
      isLineTracking = true;
      //if (isItemPicked()) {
      if(pickUpTimer == 0){
```

```
      pickUpTimer = millis();
    }
    if (millis() - pickUpTimer >= 14000) {
      pickUpPerformed = true;
      colorMatchStarted = true;
      pickUpTimer = 0;
      stepCnt = 3;
    }
    //}
  }
  break;
 case 3:
  if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
    Stop();
    alarmColorON();
    isLineTracking = false;
    colorMatchStarted = false;
  }
  else{
    alarmColorOFF();
    colorMatchStarted = true;
    isLineTracking = true;
    if(pickUpPerformed && isColorPurple && current_claw_degrees <= 20){
      isLineTracking = false;
      colorMatchStarted = false; // Reset the flag
      if(isItemPicked()){
        Stop();
        delay(200);
        drop_itemPurple();
        dropOffPerformed = true;
        isLineTracking = true;
        colorMatchStarted = true;
        stepCnt = 4;
      }
    }
  }
  break;
 case 4:
  if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
    Stop();
    alarmColorON();
    isLineTracking = false;
    colorMatchStarted = false;
  }
  else{
    alarmColorOFF();
    colorMatchStarted = true;
    isLineTracking = true;
  }
  break;
}

if(isColorRed){
 delay(150);
 Stop();
 delay(800);
 orange_purple_PickDrop_flag = false;
 stepCnt = 1;
```

```
      currentCycle =10;
      shouldReset=true;
      break;
    }
    if(isLineTracking){
      black_line();
    }

    if(colorMatchStarted){
      colorMatch();
    }
    if( HC05.available()>0){
      command = HC05.read();
      if(command == 's'){
        carState = STOPPED;
        Stop();
        HC05.println("Vehicle has been STOPPED!!");
      }
    }
  }
  else if(carState == STOPPED){
    if(HC05.available()>0){
      command = HC05.read();
      if(command == 'n'){
        currentCycle = 3;
        HC05.println("ORANGE - PURPLE CYCLE!!");
        carState = RUNNING;
      }
    }
  }
 }
}


void blue_blue_pick_drop(){
 blue_blue_PickDrop_flag = true;
 alarmColorOFF();

 bool pickUpPerformed = false;
 bool dropOffPerformed = false;
 bool isLineTracking = true;
 bool colorMatchStarted = true;

 unsigned long currentAvoidTimer = 0;
 unsigned long currentTime = 0;
 unsigned long continueTimer = 0;
 unsigned long pickUpTimer = 0; // Timer for pick-up delay
 unsigned int stopDistance = 15;
 unsigned int  stepCnt= 1;

 while(blue_blue_PickDrop_flag){
  currentCycle = 4;
  if(carState ==RUNNING){
    HC05.println("BLUE - BLUE CYCLE!!");
    Left_Tra_Value = digitalRead(7);
    Center_Tra_Value = digitalRead(8);
    Right_Tra_Value = digitalRead(A1);
    unsigned int distance_cm = sonarItem.ping_cm();
```

```
unsigned int distanceAvoid_cm = sonarAvoid.ping_cm();

if(distanceAvoid_cm <=40 && distanceAvoid_cm > 10){
  if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
    isLineTracking = false;
    Rotate_Left(85);
    delay(200);
    isLineTracking = true;
    Serial.println("if statement <= 40cm > 10cm");
  }
}
else if(distanceAvoid_cm > 40|| distanceAvoid_cm == 0){
  if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
    isLineTracking = false;
    Move_Forward(60);
    delay(100);
    isLineTracking = true;
  }
}

switch (stepCnt){
  case 1:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
      if(isColorBlue && (distance_cm <= stopDistance && distance_cm > 0) && current_claw_degrees >=50) {
        isLineTracking = false;
        colorMatchStarted = false; // Reset the flag
        if(isItemDropped()){
          Stop();
          delay(200);
          pick_itemBlue();
          pickUpPerformed = true;
          stepCnt = 2;
        }
      }
      else if(current_claw_degrees <= 20){
        stepCnt= 2;
      }
    }
    break;
  case 2:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      isLineTracking = true;
```

```
      colorMatchStarted = true;
      isLineTracking = true;
      if (isItemPicked()) {
        if(pickUpTimer == 0){
          pickUpTimer = millis();
        }
        if (millis() - pickUpTimer >= 3000) {
          pickUpPerformed = true;
          colorMatchStarted = true;
          pickUpTimer = 0;
          stepCnt = 3;
        }
      }
    }
    break;
  case 3:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
      if(pickUpPerformed && isColorBlue && current_claw_degrees <= 20){
        isLineTracking = false;
        colorMatchStarted = false; // Reset the flag
        if(isItemPicked()){
          Stop();
          delay(200);
          drop_itemBlue();
          dropOffPerformed = true;
          isLineTracking = true;
          colorMatchStarted = true;
          stepCnt = 4;
        }
      }
    }
    break;
  case 4:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
    }
    break;
}

if(isColorRed){
  delay(150);
```

```
      //pickUpPerformed = true;
      //dropOffPerformed = true;
      Stop();
      delay(800);
      orange_orange_PickDrop_flag = false;
      stepCnt = 1;
      currentCycle =10;
      shouldReset=true;
      break;
    }
    if(isLineTracking){
      black_line();
    }

    if(colorMatchStarted){
      colorMatch();
    }
    if( HC05.available()>0){
      command = HC05.read();
      if(command == 's'){
        carState = STOPPED;
        Stop();
        HC05.println("Vehicle has been STOPPED!!");
      }
    }
  }
  else if(carState == STOPPED){
    if(HC05.available()>0){
      command = HC05.read();
      if(command == 'n'){
        currentCycle = 4;
        HC05.println("BLUE - BLUE CYCLE!!");
        carState = RUNNING;
      }
    }
  }
 }
}

void blue_orange_pick_drop(){
 blue_orange_PickDrop_flag = true;
 alarmColorOFF();

 bool pickUpPerformed = false;
 bool dropOffPerformed = false;
 bool isLineTracking = true;
 bool colorMatchStarted = true;

 unsigned long currentAvoidTimer = 0;
 unsigned long currentTime = 0;
 unsigned long continueTimer = 0;
 unsigned long pickUpTimer = 0; // Timer for pick-up delay
 unsigned int stopDistance = 15;
 unsigned int  stepCnt= 1;

 while(blue_orange_PickDrop_flag){
  currentCycle = 5;
  if(carState ==RUNNING){
```

```
HC05.println("BLUE - ORANGE CYCLE!!");
Left_Tra_Value = digitalRead(7);
Center_Tra_Value = digitalRead(8);
Right_Tra_Value = digitalRead(A1);
unsigned int distance_cm = sonarItem.ping_cm();
unsigned int distanceAvoid_cm = sonarAvoid.ping_cm();

if(distanceAvoid_cm <=40 && distanceAvoid_cm > 10){
  if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
    isLineTracking = false;
    Rotate_Left(85);
    delay(200);
    isLineTracking = true;
  }
}
else if(distanceAvoid_cm > 40|| distanceAvoid_cm == 0){
  if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
    isLineTracking = false;
    Move_Forward(60);
    delay(100);
    isLineTracking = true;
    Serial.println("if statement >40cm");
  }
}

switch (stepCnt){
  case 1:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
      if(isColorBlue && (distance_cm <= stopDistance && distance_cm > 0) && current_claw_degrees >=50) {
        isLineTracking = false;
        colorMatchStarted = false; // Reset the flag
        if(isItemDropped()){
          Stop();
          delay(200);
          pick_itemBlue();
          pickUpPerformed = true;
          stepCnt = 2;
        }
      }
      else if(current_claw_degrees <= 20){
        stepCnt= 2;
      }
    }
    break;
  case 2:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
```

```
        colorMatchStarted = false;
      }
      else{
        alarmColorOFF();
        isLineTracking = true;
        colorMatchStarted = true;
        isLineTracking = true;
        if (isItemPicked()) {
          if(pickUpTimer == 0){
            pickUpTimer = millis();
          }
          if (millis() - pickUpTimer >= 2000) {
            pickUpPerformed = true;
            colorMatchStarted = true;
            pickUpTimer = 0;
            stepCnt = 3;
          }
        }
      }
      break;
    case 3:
      if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
        Stop();
        alarmColorON();
        isLineTracking = false;
        colorMatchStarted = false;
      }
      else{
        alarmColorOFF();
        colorMatchStarted = true;
        isLineTracking = true;
        if(pickUpPerformed && isColorOrange  && current_claw_degrees <= 20){ //&& (distance_cm > stopDistance ||
distance_cm <= 0)
          isLineTracking = false;
          colorMatchStarted = false; // Reset the flag
          if(isItemPicked()){
            Stop();
            delay(200);
            drop_itemOrange();
            dropOffPerformed = true;
            isLineTracking = true;
            colorMatchStarted = true;
            stepCnt = 4;
          }
        }
      }
      break;
    case 4:
      if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
        Stop();
        alarmColorON();
        isLineTracking = false;
        colorMatchStarted = false;
      }
      else{
        alarmColorOFF();
        colorMatchStarted = true;
        isLineTracking = true;
```

```
        }
        break;
      }

      if(isColorRed){
        delay(150);
        //pickUpPerformed = true;
        //dropOffPerformed = true;
        Stop();
        delay(800);
        blue_blue_PickDrop_flag = false;
        stepCnt = 1;
        currentCycle =10;
        shouldReset=true;
        break;
      }
      if(isLineTracking){
        black_line();
      }

      if(colorMatchStarted){
        colorMatch();
      }
      if( HC05.available()>0){
        command = HC05.read();
        if(command == 's'){
          carState = STOPPED;
          Stop();
          HC05.println("Vehicle has been STOPPED!!");
        }
      }
    }
    else if(carState == STOPPED){
      if(HC05.available()>0){
        command = HC05.read();
        if(command == 'n'){
          currentCycle =5;
          HC05.println("BLUE - ORANGE CYCLE!!");
          carState = RUNNING;
        }
      }
    }
  }
}


void blue_purple_pick_drop(){
  blue_purple_PickDrop_flag = true;
  alarmColorOFF();

  bool pickUpPerformed = false;
  bool dropOffPerformed = false;
  bool isLineTracking = true;
  bool colorMatchStarted = true;

  unsigned long currentAvoidTimer = 0;
  unsigned long currentTime = 0;
  unsigned long continueTimer = 0;
```

```
unsigned long pickUpTimer = 0; // Timer for pick-up delay
unsigned int stopDistance = 15;
unsigned int  stepCnt= 1;

while(blue_purple_PickDrop_flag){
  currentCycle = 6;
  if(carState == RUNNING){
    HC05.println("BLUE - PURPLE CYCLE!!");
    Left_Tra_Value = digitalRead(7);
    Center_Tra_Value = digitalRead(8);
    Right_Tra_Value = digitalRead(A1);
    unsigned int distance_cm = sonarItem.ping_cm();
    unsigned int distanceAvoid_cm = sonarAvoid.ping_cm();

    if(distanceAvoid_cm <=40 && distanceAvoid_cm > 10){
      if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
        isLineTracking = false;
        Rotate_Left(85);
        delay(200);
        isLineTracking = true;
      }
    }
    else if(distanceAvoid_cm > 40|| distanceAvoid_cm == 0){
      if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
        isLineTracking = false;
        Move_Forward(60);
        delay(100);
        isLineTracking = true;
      }
    }

    switch (stepCnt){
      case 1:
        if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
          Stop();
          alarmColorON();
          isLineTracking = false;
          colorMatchStarted = false;
        }
        else{
          alarmColorOFF();
          colorMatchStarted = true;
          isLineTracking = true;
          if(isColorBlue && (distance_cm <= stopDistance && distance_cm > 0) && current_claw_degrees >=50) {
            isLineTracking = false;
            colorMatchStarted = false; // Reset the flag
            if(isItemDropped()){
              Stop();
              delay(200);
              pick_itemBlue();
              pickUpPerformed = true;
              stepCnt = 2;
            }
          }
          else if(current_claw_degrees <= 20){
            stepCnt= 2;
          }
        }
```

```
  break;
 case 2:
  if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
    Stop();
    alarmColorON();
    isLineTracking = false;
    colorMatchStarted = false;
  }
  else{
    alarmColorOFF();
    isLineTracking = true;
    colorMatchStarted = true;
    isLineTracking = true;
    if (isItemPicked()) {
      if(pickUpTimer == 0){
        pickUpTimer = millis();
      }
      if (millis() - pickUpTimer >= 6000) {
        pickUpPerformed = true;
        colorMatchStarted = true;
        pickUpTimer = 0;
        stepCnt = 3;
      }
    }
  }
  break;
 case 3:
  if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
    Stop();
    alarmColorON();
    isLineTracking = false;
    colorMatchStarted = false;
  }
  else{
    alarmColorOFF();
    colorMatchStarted = true;
    isLineTracking = true;
    if(pickUpPerformed && isColorPurple  && current_claw_degrees <= 20){
      isLineTracking = false;
      colorMatchStarted = false; // Reset the flag
      if(isItemPicked()){
        Stop();
        delay(200);
        drop_itemPurple();
        dropOffPerformed = true;
        isLineTracking = true;
        colorMatchStarted = true;
        stepCnt = 4;
      }
    }
  }
  break;
 case 4:
  if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
    Stop();
    alarmColorON();
    isLineTracking = false;
    colorMatchStarted = false;
```

```
        }
        else{
          alarmColorOFF();
          colorMatchStarted = true;
          isLineTracking = true;
        }
        break;
      }

      if(isColorRed){
        delay(150);
        //pickUpPerformed = true;
        //dropOffPerformed = true;
        Stop();
        delay(800);
        blue_purple_PickDrop_flag = false;
        stepCnt = 1;
        currentCycle =10;
        shouldReset=true;
        break;
      }
      if(isLineTracking){
        black_line();
      }

      if(colorMatchStarted){
        colorMatch();
      }
      if( HC05.available()>0){
        command = HC05.read();
        if(command == 's'){
          carState = STOPPED;
          Stop();
          HC05.println("Vehicle has been STOPPED!!");
        }
      }
    }
    else if(carState == STOPPED){
      if(HC05.available()>0){
        command = HC05.read();
        if(command == 'n'){
          currentCycle = 6;
          HC05.println("BLUE - PURPLE CYCLE!!");
          carState = RUNNING;
        }
      }
    }
  }
}

void purple_purple_pick_drop(){
  purple_purple_PickDrop_flag = true;
  alarmColorOFF();
  bool pickUpPerformed = false;
  bool dropOffPerformed = false;
  bool isLineTracking = true;
  bool colorMatchStarted = true;
```

```
unsigned long currentAvoidTimer = 0;
unsigned long currentTime = 0;
unsigned long continueTimer = 0;
unsigned long pickUpTimer = 0; // Timer for pick-up delay
unsigned int stopDistance = 15;
unsigned int  stepCnt= 1;

while(purple_purple_PickDrop_flag){
  if(carState ==RUNNING){
    currentCycle = 7;
    HC05.println("PURPLE - PURPLE CYCLE!!");
    Left_Tra_Value = digitalRead(7);
    Center_Tra_Value = digitalRead(8);
    Right_Tra_Value = digitalRead(A1);
    unsigned int distance_cm = sonarItem.ping_cm();
    unsigned int distanceAvoid_cm = sonarAvoid.ping_cm();

    if(distanceAvoid_cm <=40 && distanceAvoid_cm > 10){
      if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
        isLineTracking = false;
        Rotate_Left(85);
        delay(200);
        isLineTracking = true;
      }
    }
    else if(distanceAvoid_cm > 40|| distanceAvoid_cm == 0){
      if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
        isLineTracking = false;
        Move_Forward(60);
        delay(100);
        isLineTracking = true;
      }
    }

    switch (stepCnt){
      case 1:
        if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
          Stop();
          alarmColorON();
          isLineTracking = false;
          colorMatchStarted = false;
        }
        else{
          alarmColorOFF();

          colorMatchStarted = true;
          isLineTracking = true;
          if(isColorPurple && (distance_cm <= stopDistance && distance_cm > 0) && current_claw_degrees >=50) {
            isLineTracking = false;
            colorMatchStarted = false; // Reset the flag
            if(isItemDropped()){
              Stop();
              delay(200);
              pick_itemPurple();
              pickUpPerformed = true;
              stepCnt = 2;
            }
          }
```

```
    else if(isItemPicked()){
      stepCnt= 2;
    }
  }
  break;
case 2:
  if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
    Stop();
    alarmColorON();
    isLineTracking = false;
    colorMatchStarted = false;
  }
  else{
    alarmColorOFF();
    isLineTracking = true;
    colorMatchStarted = true;
    isLineTracking = true;
    if (isItemPicked()) {
      if(pickUpTimer == 0){
        pickUpTimer = millis();
      }
      if (millis() - pickUpTimer >= 2000) {
        pickUpPerformed = true;
        colorMatchStarted = true;
        pickUpTimer = 0;
        stepCnt = 3;
      }
    }
  }
  break;
case 3:
  if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
    Stop();
    alarmColorON();
    isLineTracking = false;
    colorMatchStarted = false;
  }
  else{
    alarmColorOFF();

    colorMatchStarted = true;
    isLineTracking = true;
    if(pickUpPerformed && isColorPurple && current_claw_degrees <= 20 ){
      isLineTracking = false;
      colorMatchStarted = false; // Reset the flag
      if(isItemPicked()){
        Stop();
        delay(200);
        drop_itemPurple();
        dropOffPerformed = true;
        isLineTracking = true;
        colorMatchStarted = true;
        HC05.println("Step 3 complete!");
        stepCnt = 4;
      }
    }
  }
  break;
```

```
      case 4:
        if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
          Stop();
          alarmColorON();
          isLineTracking = false;
          colorMatchStarted = false;
        }
        else{
          alarmColorOFF();
          colorMatchStarted = true;
          isLineTracking = true;
        }
        break;

    }

    if(isColorRed){
      delay(150);
      //pickUpPerformed = true;
      //dropOffPerformed = true;
      Stop();
      delay(800);
      purple_purple_PickDrop_flag = false;
      stepCnt = 1;
      currentCycle =10;
      shouldReset=true;

      break;
    }
    if(isLineTracking){
      black_line();
    }

    if(colorMatchStarted){
      colorMatch();
    }
    if( HC05.available()>0){
      command = HC05.read();
      if(command == 's'){
        carState = STOPPED;
        Stop();
        HC05.println("Vehicle has been STOPPED!!");
      }
    }
  }
  else if(carState == STOPPED){
    if(HC05.available()>0){
      command = HC05.read();
      if(command == 'n'){
        stepCnt = 1;
        currentCycle = 7;
        HC05.println("PURPLE - PURPLE CONTINUE CYCLE!!");
        carState = RUNNING;
      }
    }
  }
 }
}
```

```
void purple_orange_pick_drop(){
  purple_orange_PickDrop_flag = true;
  alarmColorOFF();
  bool pickUpPerformed = false;
  bool dropOffPerformed = false;
  bool isLineTracking = true;
  bool colorMatchStarted = true;

  unsigned long currentAvoidTimer = 0;
  unsigned long currentTime = 0;
  unsigned long continueTimer = 0;
  unsigned long pickUpTimer = 0; // Timer for pick-up delay
  unsigned int stopDistance = 15;
  unsigned int  stepCnt= 1;

  while(purple_orange_PickDrop_flag){
    if(carState ==RUNNING){
      currentCycle = 8;
      HC05.println("PURPLE - ORANGE CYCLE!!");
      Left_Tra_Value = digitalRead(7);
      Center_Tra_Value = digitalRead(8);
      Right_Tra_Value = digitalRead(A1);
      unsigned int distance_cm = sonarItem.ping_cm();
      unsigned int distanceAvoid_cm = sonarAvoid.ping_cm();

      if(distanceAvoid_cm <=40 && distanceAvoid_cm > 10){
        if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
          isLineTracking = false;
          Rotate_Left(85);
          delay(200);
          isLineTracking = true;
        }
      }
      else if(distanceAvoid_cm > 40|| distanceAvoid_cm == 0){
        if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
          isLineTracking = false;
          Move_Forward(60);
          delay(100);
          isLineTracking = true;
        }
      }

      switch (stepCnt){
        case 1:
          if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
            Stop();
            alarmColorON();
            isLineTracking = false;
            colorMatchStarted = false;
          }
          else{
            alarmColorOFF();

            colorMatchStarted = true;
            isLineTracking = true;
            if(isColorPurple && (distance_cm <= stopDistance && distance_cm > 0) && current_claw_degrees >=50) {
              isLineTracking = false;
```

```
      colorMatchStarted = false; // Reset the flag
      if(isItemDropped()){
        Stop();
        delay(200);
        pick_itemPurple();
        pickUpPerformed = true;
        stepCnt = 2;
      }
    }
    else if(isItemPicked()){
      stepCnt= 2;
    }
  }
  break;
case 2:
  if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
    Stop();
    alarmColorON();
    isLineTracking = false;
    colorMatchStarted = false;
  }
  else{
    alarmColorOFF();
    isLineTracking = true;
    colorMatchStarted = true;
    isLineTracking = true;
    if (isItemPicked()) {
      if(pickUpTimer == 0){
        pickUpTimer = millis();
      }
      if (millis() - pickUpTimer >= 2000) {
        pickUpPerformed = true;
        colorMatchStarted = true;
        pickUpTimer = 0;
        stepCnt = 3;
      }
    }
  }
  break;
case 3:
  if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
    Stop();
    alarmColorON();
    isLineTracking = false;
    colorMatchStarted = false;
  }
  else{
    alarmColorOFF();
    colorMatchStarted = true;
    isLineTracking = true;
    if(pickUpPerformed && isColorOrange  && current_claw_degrees <= 20 ){
      isLineTracking = false;
      colorMatchStarted = false; // Reset the flag
      if(isItemPicked()){
        Stop();
        delay(200);
        drop_itemOrange();
        dropOffPerformed = true;
```

```
          isLineTracking = true;
          colorMatchStarted = true;
          stepCnt = 4;
        }
      }
    }
    break;
  case 4:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
    }
    break;

  }

  if(isColorRed){
    delay(150);
    //pickUpPerformed = true;
    //dropOffPerformed = true;
    Stop();
    delay(800);
    purple_orange_PickDrop_flag = false;
    stepCnt = 1;
    currentCycle =10;
    shouldReset=true;

    break;
  }
  if(isLineTracking){
    black_line();
  }

  if(colorMatchStarted){
    colorMatch();
  }
  if( HC05.available()>0){
    command = HC05.read();
    if(command == 's'){
      carState = STOPPED;
      Stop();
      HC05.println("Vehicle has been STOPPED!!");
    }
  }
}
else if(carState == STOPPED){
  if(HC05.available()>0){
    command = HC05.read();
    if(command == 'n'){
      stepCnt = 1;
      currentCycle = 8;
```

```
        HC05.println("PURPLE - ORANGE CONTINUE CYCLE!!");
        carState = RUNNING;
      }
    }
  }
 }
}

void purple_blue_pick_drop(){
 purple_blue_PickDrop_flag = true;
 alarmColorOFF();

 bool pickUpPerformed = false;
 bool dropOffPerformed = false;
 bool isLineTracking = true;
 bool colorMatchStarted = true;

 unsigned long currentAvoidTimer = 0;
 unsigned long currentTime = 0;
 unsigned long continueTimer = 0;
 unsigned long pickUpTimer = 0; // Timer for pick-up delay
 unsigned int stopDistance = 15;
 unsigned int  stepCnt= 1;

 while(purple_blue_PickDrop_flag){
  currentCycle = 9;
  if(carState == RUNNING){
    HC05.println("PURPLE - BLUE CYCLE!!");
    Left_Tra_Value = digitalRead(7);
    Center_Tra_Value = digitalRead(8);
    Right_Tra_Value = digitalRead(A1);
    unsigned int distance_cm = sonarItem.ping_cm();
    unsigned int distanceAvoid_cm = sonarAvoid.ping_cm();

    if(distanceAvoid_cm <=40 && distanceAvoid_cm > 10){
      if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
        isLineTracking = false;
        Rotate_Left(85);
        delay(200);
        isLineTracking = true;
        Serial.println("if statement <= 40cm > 10cm");
      }
    }
    else if(distanceAvoid_cm > 40|| distanceAvoid_cm == 0){
      if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
        isLineTracking = false;
        Move_Forward(60);
        delay(100);
        isLineTracking = true;
        Serial.println("if statement >40cm");
      }
    }

    switch (stepCnt){
      case 1:
        if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
          Stop();
          alarmColorON();
```

```
    isLineTracking = false;
    colorMatchStarted = false;
  }
  else{
    alarmColorOFF();
    colorMatchStarted = true;
    isLineTracking = true;
    if(isColorPurple && (distance_cm <= stopDistance && distance_cm > 0) && current_claw_degrees >=50) {
      isLineTracking = false;
      colorMatchStarted = false; // Reset the flag
      if(isItemDropped()){
        Stop();
        delay(200);
        pick_itemPurple();
        pickUpPerformed = true;
        stepCnt = 2;
      }
    } else if(current_claw_degrees <= 20){
      stepCnt= 2;
    }
  }
  break;
case 2:
  if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
    Stop();
    alarmColorON();
    isLineTracking = false;
    colorMatchStarted = false;
  }
  else{
    alarmColorOFF();
    isLineTracking = true;
    colorMatchStarted = true;
    isLineTracking = true;
    if (isItemPicked()) {
      if(pickUpTimer == 0){
        pickUpTimer = millis();
      }
      if (millis() - pickUpTimer >= 2000) {
        pickUpPerformed = true;
        colorMatchStarted = true;
        pickUpTimer = 0;
        stepCnt = 3;
      }
    }
  }
  break;
case 3:
  if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
    Stop();
    alarmColorON();
    isLineTracking = false;
    colorMatchStarted = false;
  }
  else{
    alarmColorOFF();
    colorMatchStarted = true;
    isLineTracking = true;
```

```
      if(pickUpPerformed && isColorBlue && current_claw_degrees <= 20){
        isLineTracking = false;
        colorMatchStarted = false; // Reset the flag
        if(isItemPicked()){
          Stop();
          delay(200);
          drop_itemBlue();
          dropOffPerformed = true;
          isLineTracking = true;
          colorMatchStarted = true;
          stepCnt = 4;
        }
      }
    }
    break;
  case 4:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
    }
    break;
  }

  if(isColorRed){
    delay(150);
    //pickUpPerformed = true;
    //dropOffPerformed = true;
    Stop();
    delay(800);
    purple_blue_PickDrop_flag = false;
    stepCnt = 1;
    currentCycle =10;
    shouldReset=true;
    break;
  }
  if(isLineTracking){
    black_line();
  }

  if(colorMatchStarted){
    colorMatch();
  }
  if( HC05.available()>0){
    command = HC05.read();
    if(command == 's'){
      carState = STOPPED;
      Stop();
      HC05.println("Vehicle has been STOPPED!!");
    }
  }
} else if(carState == STOPPED){
```

```
      if(HC05.available()>0){
        command = HC05.read();
        if(command == 'n'){
          currentCycle = 9;
          HC05.println("PURPLE - BLUE CYCLE!!");
          carState = RUNNING;
        }
      }
    }
  }
}

//These functions jobOne, jobTwo, and jobThree will be towards the simulation cycle that start with joOne() and ends at
jobThree().
//these functions are similar to the multicolor function with only thing that must loop through all three
//functions to complete the cycles.
void jobOne() {
  alarmColorOFF();
  jobOne_flag = true;
  jobCycleCounter = 1;

  bool pickUpPerformed = false; // Flag to track whether pick-up action has been performed
  bool dropOffPerformed = false;
  bool isLineTracking = true;
  bool colorMatchStarted = true;

  unsigned long currentAvoidTimer = 0;
  unsigned long currentTime = 0;
  unsigned long continueTimer = 0;
  unsigned long pickUpTimer = 0; // Timer for pick-up delay
  unsigned int stopDistance = 15;

  unsigned int  stepCnt= 1;
  while(jobOne_flag && jobCycleCounter == 1) {
    currentCycle = 11;
    if(carState == RUNNING){
      HC05.println("CYCLE 1: ORANGE - ORANGE STATIONS!!");
      Left_Tra_Value = digitalRead(7);
      Center_Tra_Value = digitalRead(8);
      Right_Tra_Value = digitalRead(A1);
      unsigned int distance_cm = sonarItem.ping_cm();
      unsigned int distanceAvoid_cm = sonarAvoid.ping_cm();

      if(distanceAvoid_cm <=40 && distanceAvoid_cm > 10){
        if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
          isLineTracking = false;
          Rotate_Left(75);
          delay(200);
          isLineTracking = true;
          Serial.println("if statement <= 40cm > 10cm");
        }
      }
      else if(distanceAvoid_cm > 40|| distanceAvoid_cm == 0){
        if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
          isLineTracking = false;
          Move_Forward(60);
          delay(100);
          isLineTracking = true;
```

```
  }
}

switch (stepCnt){
  case 1:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
      if(isColorOrange && (distance_cm <= stopDistance && distance_cm > 0) && !pickUpPerformed) {
        isLineTracking = false;
        colorMatchStarted = false; // Reset the flag
        if(isItemDropped()){
          Stop();
          delay(200);
          pick_itemOrange();
          stepCnt = 2;
        }
      }else if(current_claw_degrees <= 20){
        stepCnt= 2;
      }
    }
    break;
  case 2:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      isLineTracking = true;
      colorMatchStarted = true;
      isLineTracking = true;
      if (isItemPicked()) {
        if(pickUpTimer == 0){
          pickUpTimer = millis();
        }
        if (millis() - pickUpTimer >= 3000) {
          pickUpPerformed = true;
          colorMatchStarted = true;
          pickUpTimer = 0;
          stepCnt = 3;
        }
      }
    }
    break;
  case 3:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
```

```
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
      if(pickUpPerformed && isColorOrange && current_claw_degrees <= 20 ){
        isLineTracking = false;
        colorMatchStarted = false; // Reset the flag
        if(isItemPicked()){
          Stop();
          delay(200);
          drop_itemOrange();
          dropOffPerformed = true;
          isLineTracking = true;
          colorMatchStarted = true;
          stepCnt = 4;
        }
      }
    }
    break;
  case 4:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
    }
    break;
  }

if(isColorRed){
  delay(150);
  Stop();
  delay(800);
  isLineTracking = true;
  colorMatchStarted = true;
  jobCycleCounter = 2;
  jobOne_flag = false;
  currentCycle = 12;
}
if(isLineTracking){
  black_line();
}

if(colorMatchStarted){
  colorMatch();
}
if( HC05.available()>0){
  command = HC05.read();
  if(command == 's'){
    carState = STOPPED;
```

```
        Stop();
        HC05.println("Vehicle has been STOPPED!!");
      }
    }
  }
  else if(carState == STOPPED){
    if(HC05.available()>0){
      command = HC05.read();
      if(command == 'n'){
        currentCycle = 11;
        HC05.println("CYCLE 1: ORANGE - ORANGE STATIONS!!");
        carState = RUNNING;
      }
    }
  }
 }
}

void jobTwo() {
 jobTwo_flag = true;
 alarmColorOFF();
 bool pickUpPerformed = false; // Flag to track whether pick-up action has been performed
 bool dropOffPerformed = false;
 bool isLineTracking = true;
 bool colorMatchStarted = true;

 unsigned long currentAvoidTimer = 0;
 unsigned long currentTime = 0;
 unsigned long continueTimer = 0;
 unsigned long pickUpTimer = 0; // Timer for pick-up delay
 unsigned int stopDistance = 15;
 unsigned int  stepCnt= 1;

 while(jobTwo_flag && jobCycleCounter == 2) {
  currentCycle = 12;
  if(carState == RUNNING){
    HC05.println("CYCLE 2: BLUE - BLUE STATIONS!!");
    Left_Tra_Value = digitalRead(7);
    Center_Tra_Value = digitalRead(8);
    Right_Tra_Value = digitalRead(A1);
    unsigned int distance_cm = sonarItem.ping_cm();
    unsigned int distanceAvoid_cm = sonarAvoid.ping_cm();
    if(distanceAvoid_cm <=40 && distanceAvoid_cm > 10){
      if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
        isLineTracking = false;
        Rotate_Left(75);
        delay(200);
        isLineTracking = true;
        Serial.println("if statement <= 40cm > 10cm");
      }
    }
    else if(distanceAvoid_cm > 40|| distanceAvoid_cm == 0){
      if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
        isLineTracking = false;
        Move_Forward(60);
        delay(100);
        isLineTracking = true;
        Serial.println("if statement >40cm");
```

```
  }
}
switch (stepCnt){
  case 1:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
      if(isColorBlue && (distance_cm <= stopDistance && distance_cm > 0) && !pickUpPerformed) {
        isLineTracking = false;
        colorMatchStarted = false; // Reset the flag
        if(isItemDropped()){
          Stop();
          delay(200);
          pick_itemBlue();
          stepCnt = 2;
        }
      }else if(current_claw_degrees <= 20){
        stepCnt= 2;
      }
    }
    break;
  case 2:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      isLineTracking = true;
      colorMatchStarted = true;
      isLineTracking = true;
      if (isItemPicked()) {
        if(pickUpTimer == 0){
          pickUpTimer = millis();
        }
        if (millis() - pickUpTimer >= 3000) {
          pickUpPerformed = true;
          colorMatchStarted = true;
          pickUpTimer = 0;
          stepCnt = 3;
        }
      }
    }
    break;
  case 3:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
```

```
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
      if(pickUpPerformed && isColorBlue && current_claw_degrees <= 20 ){
        isLineTracking = false;
        colorMatchStarted = false; // Reset the flag
        if(isItemPicked()){
          Stop();
          delay(200);
          drop_itemBlue();
          dropOffPerformed = true;
          isLineTracking = true;
          colorMatchStarted = true;
          stepCnt = 4;
        }
      }
    }
    break;
  case 4:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
    }
    break;
}

if(isColorRed){
  delay(150);
  Stop();
  delay(800);
  currentCycle = 13;
  jobCycleCounter = 3;

}
if(isLineTracking){
  black_line();
}

if(colorMatchStarted){
  colorMatch();
}
if( HC05.available()>0){
  command = HC05.read();
  if(command == 's'){
    carState = STOPPED;
    Stop();
    HC05.println("Vehicle has been STOPPED!!");
  }
```

```
      }
    }
    else if(carState == STOPPED){
      if(HC05.available()>0){
        command = HC05.read();
        if(command == 'n'){
          currentCycle = 12;
          HC05.println("CYCLE 2: BLUE - BLUE STATIONS!!");
          carState = RUNNING;
        }
      }
    }
  }
}

void jobThree() {
  jobThree_flag = true;
  alarmColorOFF();
  bool pickUpPerformed = false; // Flag to track whether pick-up action has been performed
  bool dropOffPerformed = false;
  bool isLineTracking = true;
  bool colorMatchStarted = true;

  unsigned long currentAvoidTimer = 0;
  unsigned long currentTime = 0;
  unsigned long continueTimer = 0;
  unsigned long pickUpTimer = 0; // Timer for pick-up delay
  unsigned int stopDistance = 15;
  unsigned int  stepCnt= 1;

  while(jobThree_flag && jobCycleCounter == 3) {
    currentCycle = 13;
    if(carState == RUNNING){
      HC05.println("CYCLE 3: PURPLE - PURPLE STATIONS!!");
      Left_Tra_Value = digitalRead(7);
      Center_Tra_Value = digitalRead(8);
      Right_Tra_Value = digitalRead(A1);
      unsigned int distance_cm = sonarItem.ping_cm();
      unsigned int distanceAvoid_cm = sonarAvoid.ping_cm();

      if(distanceAvoid_cm <=40 && distanceAvoid_cm > 10){
        if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
          isLineTracking = false;
          Rotate_Left(75);
          delay(200);
          isLineTracking = true;
          Serial.println("if statement <= 40cm > 10cm");
        }
      }
      else if(distanceAvoid_cm > 40|| distanceAvoid_cm == 0){
        if(Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line){
          isLineTracking = false;
          Move_Forward(60);
          delay(100);
          isLineTracking = true;
          Serial.println("if statement >40cm");
        }
      }
```

```
switch (stepCnt){
  case 1:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
      if(isColorPurple && (distance_cm <= stopDistance && distance_cm > 0) && !pickUpPerformed) {
        isLineTracking = false;
        colorMatchStarted = false; // Reset the flag
        if(isItemDropped()){
          Stop();
          delay(200);
          pick_itemPurple();
          stepCnt = 2;
        }
      }else if(current_claw_degrees <= 20){
        stepCnt= 2;
      }
    }
    break;
  case 2:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      isLineTracking = true;
      colorMatchStarted = true;
      isLineTracking = true;
      if (isItemPicked()) {
        if(pickUpTimer == 0){
          pickUpTimer = millis();
        }
        if (millis() - pickUpTimer >= 3000) {
          pickUpPerformed = true;
          colorMatchStarted = true;
          pickUpTimer = 0;
          stepCnt = 3;
        }
      }
    }
    break;
  case 3:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
```

```
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
      if(pickUpPerformed && isColorPurple && current_claw_degrees <= 20 ){
        isLineTracking = false;
        colorMatchStarted = false; // Reset the flag
        if(isItemPicked()){
          Stop();
          delay(200);
          drop_itemPurple();
          dropOffPerformed = true;
          isLineTracking = true;
          colorMatchStarted = true;
          stepCnt = 4;
        }
      }
    }
    break;
  case 4:
    if (distanceAvoid_cm <= 10 && distanceAvoid_cm > 0) {
      Stop();
      alarmColorON();
      isLineTracking = false;
      colorMatchStarted = false;
    }
    else{
      alarmColorOFF();
      colorMatchStarted = true;
      isLineTracking = true;
    }
    break;
}

if(isColorRed){
  delay(150);
  Stop();
  delay(800);
  currentCycle = 10;
  jobCycleCounter = 4;
  jobThree_flag = false;
  break;
}
if(isLineTracking){
  black_line();
}

if(colorMatchStarted){
  colorMatch();
}
if( HC05.available()>0){
  command = HC05.read();
  if(command == 's'){
    carState = STOPPED;
    Stop();
    HC05.println("Vehicle has been STOPPED!!");
  }
```

```
      }
    }
    else if(carState == STOPPED){
      if(HC05.available()>0){
        command = HC05.read();
        if(command == 'n'){
          currentCycle = 13;
          HC05.println("CYCLE 3: PURPLE - PURPLE STATIONS!!");
          carState = RUNNING;
        }
      }
    }
  }
}


//the black line function to track the vehicle and keep it steady and run on the black route only.
void black_line(){
  Left_Tra_Value = digitalRead(7);
  Center_Tra_Value = digitalRead(8);
  Right_Tra_Value = digitalRead(A1);
    if (Left_Tra_Value != Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value != Black_Line) {
      Move_Forward(60);
    } else if (Left_Tra_Value == Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value != Black_Line) {
      Rotate_Left(50);
    } else if (Left_Tra_Value == Black_Line && Center_Tra_Value != Black_Line && Right_Tra_Value != Black_Line) {
      Rotate_Left(50);
    } else if (Left_Tra_Value != Black_Line && Center_Tra_Value != Black_Line && Right_Tra_Value == Black_Line) {
      Rotate_Right(50);
    } else if (Left_Tra_Value != Black_Line && Center_Tra_Value == Black_Line && Right_Tra_Value == Black_Line) {
      Rotate_Right(50);
    } else if(Center_Tra_Value != Black_Line){
      Rotate_Right(40);
      Rotate_Left(40);
    }
}

//these functions are used to move the vehicle on the direction required, some are retrieved from LAFVIN coding.
void Move_Forward(int speed) {
  digitalWrite(2, LOW);   // 2 DIR pin rear motors
  analogWrite(5, speed); // 5 PWM pin rear motors
  digitalWrite(4, HIGH); // 4 DIR pin front motors
  analogWrite(6, speed); // 6 PWM pin front motors
}

void Move_Backward(int speed) {
  digitalWrite(2, HIGH);
  analogWrite(5, speed);
  digitalWrite(4, LOW);
  analogWrite(6, speed);
}

void Rotate_Left(int speed) {
  digitalWrite(2, LOW);
  analogWrite(5, speed);
  digitalWrite(4, LOW);
  analogWrite(6, speed);
}
```

```
void Rotate_Right(int speed) {
  digitalWrite(2, HIGH);
  analogWrite(5, speed);
  digitalWrite(4, HIGH);
  analogWrite(6, speed);
}

void Stop() {
  digitalWrite(2, HIGH);
  analogWrite(5, 0);
  digitalWrite(4, LOW);
  analogWrite(6, 0);
}

//these functions are for robot arm movements and functionality using servo motors
void move_claw(int degrees) {
  //this will map the degree so it is only between 0-180 degrees
  int claw_degrees = map(degrees, 0, 180, SERVO1MIN, SERVO1MAX);
  int current_degrees = claw_degrees;
  //getting the pin anc value set to channel 0 that is used for claw
  pwm.setPWM(0,0,claw_degrees);
  int increment = 2;
  if (claw_degrees > current_degrees) {
    for (int i = current_degrees; i <= claw_degrees; i += increment) {
      pwm.setPWM(0,0,i);
      delay(10);
    }
  } else if (claw_degrees < current_degrees) {
    for (int i = current_degrees; i >= claw_degrees; i -= increment) {
      pwm.setPWM(0,0,i);
      delay(10);
    }
  }
  current_claw_degrees = degrees;

}

//move the arm up and down
void move_arm(int degrees) {
  int arm_degrees = map(degrees, 0, 180, SERVO2MIN, SERVO2MAX);
  int current_degrees = arm_degrees;
  pwm.setPWM(1,0,arm_degrees);

  int increment = 2;
  if (arm_degrees > current_degrees) {
    for (int i = current_degrees; i <= arm_degrees; i += increment) {
      pwm.setPWM(1,0,i);
      delay(10);
    }
  } else if (arm_degrees < current_degrees) {
    for (int i = current_degrees; i >= arm_degrees; i -= increment) {
      pwm.setPWM(1,0,i);
      delay(10);
    }
  }

}
```

```
//rotate the base of the robot arm right and left
void base_movement(int degrees) {
  int base_degrees = map(degrees, 0, 180, SERVO3MIN, SERVO3MAX);
  int current_degrees = base_degrees;
  pwm.setPWM(2,0,base_degrees);

  int increment = 2;
  if (base_degrees > current_degrees) {
    for (int i = current_degrees; i <= base_degrees; i += increment) {
      pwm.setPWM(2,0,i);
      delay(10);
    }
  } else if (base_degrees < current_degrees) {
    for (int i = current_degrees; i >= base_degrees; i -= increment) {
      pwm.setPWM(2,0,i);
      delay(10);
    }
  }

}

//this will reset Arduino values to default from the command taken from Python with 'g'
void resetProgram(){
  HC05.println("ALL OPERATIONS RESETTED!!");
  jobOne_flag = false;
  jobTwo_flag = false;
  jobThree_flag = false;
  stop_flag = false;
  alarm_flag = false;
  isColorOrange = false;
  isColorGreen = false;
  isColorYellow = false;
  isColorLightBlue = false;
  isColorDarkBlue = false;
  isColorBlue = false;
  isColorDarkOrange = false;
  isColorPurple =false;
  isColorPink = false;
  isColorRed = false;
  jobCycleCounter = 1;
  base_degrees = 83;
  arm_degrees = 30;
  claw_degrees = 60;
  forward_flag = false;
  backward_flag = false;
  left_flag = false;
  right_flag = false;
  isLineTracking = true;
  Left_Tra_Value = 1;
  Center_Tra_Value = 1;
  Right_Tra_Value = 1;
  Black_Line = 1;
  colorPassCnt = 0;
  command = 's';

  move_claw(claw_degrees);
  delay(500);
```

```
  move_arm(arm_degrees);
  delay(500);
  base_movement(base_degrees);
  delay(500);
  Stop();

}

//the main function of Arduino program.
void setup() {
  //initialize and set all values and pins in this function
  Wire.begin();

  // Initialize the TCS34725 RGB color sensor
  if (tcs.begin()) {
  //Serial.println("Found sensor");
  } else {
    Serial.println("No TCS34725 found ... check your connections");
     while (1); // halt!
  }

  jobOne_flag = false;
  jobTwo_flag = false;
  jobThree_flag = false;
  stop_flag = false;
  alarm_flag = false;
  isColorOrange = false;
  isColorGreen = false;
  isColorYellow = false;
  isColorLightBlue = false;
  isColorDarkBlue = false;
  isColorBlue = false;
  isColorDarkOrange = false;
  isColorPurple =false;
  isColorPink = false;
  isColorRed = false;
  jobCycleCounter = 1;
  base_degrees = 83;
  arm_degrees = 30;
  claw_degrees = 60;
  forward_flag = false;
  backward_flag = false;
  left_flag = false;
  right_flag = false;
  isLineTracking = true;
  Left_Tra_Value = 1;
  Center_Tra_Value = 1;
  Right_Tra_Value = 1;
  Black_Line = 1;
  colorPassCnt = 0;
  command = 's';
  //initialize the servo motor driver board
  pwm.begin();
  pwm.setOscillatorFrequency(27000000);
  pwm.setPWMFreq(SERVO_FREQ);  // Analog servos run at ~50 Hz updates
  delay(10);

  //initialize the servo motors angles
```

```
  move_claw(claw_degrees);
  delay(500);
  move_arm(arm_degrees);
  delay(500);
  base_movement(base_degrees);
  delay(500);
  Stop();

  //assign the pins as either input or output
  pinMode(DIRA_PIN, OUTPUT);
  pinMode(PWMA_PIN, OUTPUT);
  pinMode(DIRB_PIN, OUTPUT);
  pinMode(PWMB_PIN, OUTPUT);
  pinMode(7, INPUT); //tracking sensor center
  pinMode(8, INPUT);  //tracking sensor left
  pinMode(A1, INPUT); //tracking sensor right

  alarmColorOFF(); //set the LED alarm to off

  pinMode(TRIGGER_PIN, OUTPUT);//A4 pin for this ultrasonic sensor
  pinMode(ECHO_PIN, INPUT); // A5 pin for this ultrasonic sensor
  pinMode(TRIGGER_AVOID_PIN, OUTPUT);
  pinMode(ECHO_AVOID_PIN, INPUT);

  HC05.begin(9600);//Bluetooth serial communication Baud rate
  Serial.begin(9600); //USB serial communication Baud rate

}

//Main loop function used in Arduino
void loop() {
  //reading the command as available from Python communication serial port HC05
  if (HC05.available() > 0) {
    command = HC05.read();
    processCommand(command);

  }
  //this is to reset the cycle if needed
  if (shouldReset) {
    resetProgram();
    shouldReset = false; // Reset the flag to prevent continuous resets
  }
}
//this function is to determine what command to go to where as it receives it,
void processCommand(char command) {
  // Handle the received command
  Serial.print("Data Received: ");
  Serial.println(command);
  switch (command) {
    case 'o':
      orange_orange_pick_drop();
      break;
    case 'b':
      orange_blue_pick_drop();
      break;
    case 'p':
      orange_purple_pick_drop();
      break;
```

```
case 'e':
  blue_blue_pick_drop();
  break;
case 'l':
  blue_orange_pick_drop();
  break;
case 'm':
  blue_purple_pick_drop();
  break;
case 'a':
  purple_purple_pick_drop();
  break;
case 'c':
  purple_orange_pick_drop();
  break;
case 'd':
  purple_blue_pick_drop();
  break;

case 'i':
  //used for three cycle loop of jobs 1-3
  while(jobCycleCounter != 4){
    if(jobCycleCounter ==1){
      HC05.println("Starting Simulation 1, ORANGE - ORANGE!!");
      jobOne();
    }else if(jobCycleCounter ==2){
      HC05.println("Starting Simulation 2, BLUE - BLUE!!");
      jobTwo();
    } else if (jobCycleCounter ==3){
      HC05.println("Starting Simulation 3, PURPLE - PURPLE!!");
      jobThree();
    }else{
      break;
    }
  }
    if(jobCycleCounter == 4){
      Stop();
      shouldReset = true;
      break;
  }
  break;
case 's':
  HC05.println("Vehicle has been STOPPED!!");
  Stop();
  delay(50);
  break;
case 'g':
  HC05.println("ALL OPERATIONS COMPLETED AND RESETTED!!");
  shouldReset = true;
  break;
case 'x':
  HC05.write('y');
  break;
case 'n':
  //this will send n to only function that is running at that moment
  //HC05.println("CONTINUE - CYCLE!!");
  while(currentCycle != 10){
```

```
      switch (currentCycle){
        case 1:
          orange_orange_pick_drop();
          break;
        case 2:
          orange_blue_pick_drop();
          break;
        case 3:
          orange_purple_pick_drop();
          break;
        case 4:
          blue_blue_pick_drop();
          break;
        case 5:
          blue_orange_pick_drop();
          break;
        case 6:
          blue_purple_pick_drop();
          break;
        case 7:
          purple_purple_pick_drop();
          break;
        case 8:
          purple_orange_pick_drop();
          break;
        case 9:
          purple_blue_pick_drop();
          break;
        case 11:
          jobOne();
          break;
        case 12:
          jobTwo();
          break;
        case 13:
          jobThree();
          break;
        case 10:
          break;
      }
    }
    break;
  }
}
```

Logbook

September 1st – September 7th (Proposal Phase)

During our first week, our team had discussed various ideas which included creating a smart smoke detector which could be controlled using a phone application, creating a miniature game console using Arduino, and creating an autonomous vehicle which had the ability to transfer package contents to various locations. With the smoke detector, our team believed that this was an excellent idea; however, we did not feel our expertise and degree experience was able to make a project like this possible, especially because of the limited time constraint and amount of research required to successfully create this product. While we felt comfortable creating retro games that would have the ability to be played on a miniature Arduino console, our research proved that the type of displays available for Arduino devices were not practical for a gaming console. We found that these displays lacked the number of colors needed for our playable games and lacked screen capacity. Specifically, when researching the displays available for the hardware, these mainly ranged from ~1.8 inches to ~8 inches diagonally. Lastly, when we viewed our options for creating an autonomous vehicle, our team believed that this project had a lot of flexibility and scalability, especially because there were numerous sensors and hardware components that could be added to the vehicle. Furthermore, this project was able to incorporate the C++ programming language that was used throughout a major portion of our degree, allowing us to utilize the course materials available to us. Lastly, we were certain that this project was able to be completed in a reasonable timeframe because of our familiarity when it came to how the different pins interact with the circuit board. When meeting with our project advisor, Professor Sunda, he fully supported our idea and gave us insight, allowing us to understand what the warehouse environment he worked in was like.

September 8th – September 14th (Ordering Hardware)

During our second week, we were able to successfully create a list of functional requirements to meet our autonomous vehicle's needs which allowed us to better understand the different types of hardware that we would need to purchase for this project. Specifically, our team needed to research different vehicles that would act as a base for our project. Laith previously purchased an Arduino vehicle off Amazon which served our purpose perfectly, especially because it had the ability to handle objects and was able to be constructed from scratch. This specific vehicle was the LAFVIN Mechanical 4WD Robot Arm Smart Car Kit. After doing more research on the vehicle, we learned that it took advantage of the Arduino software which could use the C++ Programming Language. we needed to define a workable environment that the vehicle could move in. Because we needed to manipulate the environment that the vehicle was in, we decided to purchase some foam boards that served as our warehouse environment. These boards were sturdy and allowed us to create the environment and determine the path the vehicle would travel though. This idea allowed us to understand that we should incorporate safety features within the vehicle itself, which was the included ultrasonic sensors were extremely beneficial. Using this knowledge, we planned on creating two paths that allowed our vehicle to check if there is a blockage in the main path. Afterward, we discussed the different ways the vehicle was able to perform operations, and our team concluded that purchasing a RGB color sensor was necessary. Each color would serve its own unique function and allow us to define the package location and where it needs to be delivered to.

September 15th – September 21st (Building the Arduino vehicle)

During our third week, we received the packaging of our vehicle and needed to construct it. One important factor that we didn't know was that there were no instructions provided and

needed to locate them on the manufacturer's website. After carefully viewing all the information available and scanning the website, our group could not find a set of clear instructions that explained how to build the vehicle. Instead, we were able to view a guide on YouTube explaining the construction of the vehicle step-by-step. Unfortunately, the videos presented were not English speakers and carefully went through the visual demonstration. This forced our team to dedicate the week toward the creation of the vehicle and no other activities were able to be accomplished. Thankfully, we were able to verify the functionality of the vehicle and calibrate it using the included tools to ensure that the vehicle could handle miniature boxes that were 3D printed from scratch.

September 22nd – September 28th (Completing Requirement to Follow Black Line)

During our fourth week, we were able to construct the specific layout of our warehouse environment on the foam board and began coding the vehicle using the Arduino software. Throughout this productive week, we were able to successfully have our vehicle follow a black line which was a major accomplishment for the project. We also realized that when trying to use the RGB color sensors and when grabbing objects, we would need to adjust the design and incorporate our own ideas to resolve issues. Some issues included:

- Having the vehicle shake slightly when moving which affected how accurate the color reading was when approaching a particular color.
- Claw would occasionally drop objects and found the grip was weak.
- Plastic parts that held the claw were breaking and needed a replacement.

While our team has not resolved these issues this week, these were important issues that needed to get resolved before moving forward with the code creation. Nonetheless, we were able to

successfully achieve one of our functional requirements which was following the black line which served as our vehicle's path. We also purchased better motors to allow for a stronger grip.

September 29th – October 5th (Performing Maintenance on Vehicle)

During our fifth week, our team was able to resolve the three issues above. When resolving the color sensor issue, we were able to create an enclosure for it so that the color readings were more consistent (we found that the environmental lighting affected our color readings greatly). Also, we were able to create a stronger grip by adding a foam-like material to each claw, so that the boxes were able to be picked up and held onto. Because this was only a small remedy to the issue, we were able to implement better motors, so that the object doesn't drop when inside the claw (allows for limited movement of the object). Regarding the plastic pieces that were breaking, we were able to use our 3D printer again to print a replaceable piece that was more robust. For the majority of this week, we did some reparations to our vehicle, eliminating future issues; however, we were able to successfully pick up and place down boxes in the correct location but would encounter many errors when it came to different colors/stations. We were also able to work on the obstacle avoidance/detection which is our most important safety feature.

October 6th – October 12th (Discussing Obstacle Detection with Professor Sunda)

During our sixth week, we were able to successfully finish the obstacle avoidance/detection and found that it works flawlessly. However, when meeting with Professor Sunda on October 11th, we were able to show our progress of our vehicle performing its first lap. After we showed the successfully delivery of one box, the obstacle avoidance/detection, and how the alternative paths work. He encouraged us to adjust the obstacle detection/avoidance, so that instead of halting all vehicle operations indefinitely, implement a period where the vehicle

checks to see if the obstacle still exists and proceeds if removed. This was a great suggestion, especially because there may not always be a worker available to troubleshoot the vehicle, and instead, would be best to perform self-diagnostics first. We encountered issues with our other two laps (each lap moves one package to its unique area) but believe that we made good progress with our project so far.

October 13th – October 19th (Completing Obstacle Avoidance, rectifying detection)

During our seventh week, we almost managed to incorporate Professor Sunda's request with the obstacle detection but found that the vehicle moves in bursts when there is an object detected. We were able to correct this error when modifying our existing code and performed troubleshooting methods. We also met with Professor Sunda again on October 18th to discuss the closing of our project and what future weeks are going to contain. Additionally, he discussed the various industries that our vehicle would be applicable to.

October 20th – October 26th (Completing Obstacle Detection)

During our eighth week, we have rectified the obstacle detection entirely and ensured that the appropriate decisions are being made when an obstructive object is placed in front of the vehicle. We performed numerous tests to ensure that we obtained the correct result and monitored the actions that were taken. Throughout this process, we encountered an error when it came to taking the alternative route after having an object obstruct the vehicle's path, but this was corrected after we adjusted our code when taking the alternative route. We should almost have full functionality of our project and may need to tweak some portions of our code later to ensure its optimization.

October 27th – November 2nd (Testing Functional Requirements)

During our ninth week, we have ensured that all our functional requirements are met and

rectified all bugs that existed within our product. While we would like to add more functionality to our product, we decided to take this time to ensure that we have a smooth delivery of our product along with a well-created presentation.

November 3rd – November 9th (Condensing/Commenting Vehicle Code/Meeting with Advisor)

During this week, we have reformatted our code and ensured that we have it in its condensed form. We added comments and clarity so that other users can understand the events that are occurring. The code still performs correctly and tested for additional issues. Our team also met with Professor Sunda who reviewed our entire project and gave his insight on what we have accomplished. We also discussed important concepts of our presentation, allowing us to plan for it.

November 10th – November 16th (Creating PowerPoint for Vehicle/Meeting with Professor Grogan)

During this week, we started our presentation which will discuss our autonomous vehicle's hardware and software along with a video demonstration that present our findings. Furthermore, we ensured that the formatting of the presentation is optimal and allows for a professional presentation format. Furthermore, our team was able to schedule a meeting with Professor Grogan to preview our research paper and gain valuable insight. During our meeting, we discussed writing techniques that were applied to our paper and the importance of having all aspects of our project included. This meeting was critical as it allowed our team to understand the improvements that could be made to the paper and allowed us to understand where we stand with the project altogether.

November 17th – November 23rd (Adding Content to Research Paper)

This week has been reserved for completing our research paper.

November 24th – November 30th (Adding Content to Research Paper)

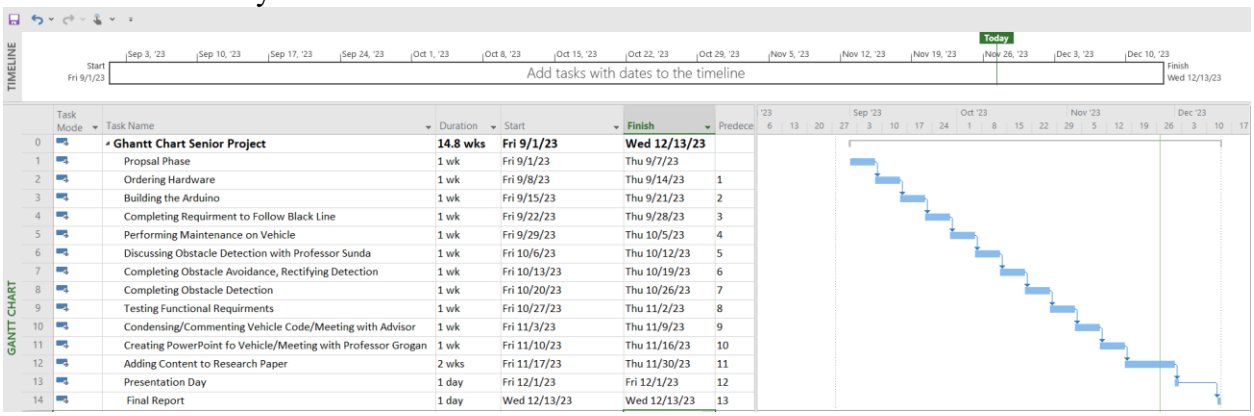
This week has been reserved for completing our research paper.

December 1st (Presentation Day)

This is the day that our team will present our project to the class.

Progress Updates

There are numerous updates that occurred throughout the creation of our autonomous delivery vehicle. While the structure of our project was undetermined when deciding on a project idea, we quickly began to brainstorm future tasks and successfully build the approach needed to complete our project. Table 1 elaborates on all activities that occurred throughout the creation of our autonomous delivery vehicle.



	Task Mode	Task Name	Duration	Start	Finish	Predece
0		**Ghantt Chart Senior Project**	**14.8 wks**	**Fri 9/1/23**	**Wed 12/13/23**	
1		Propsal Phase	1 wk	Fri 9/1/23	Thu 9/7/23	
2		Ordering Hardware	1 wk	Fri 9/8/23	Thu 9/14/23	1
3		Building the Arduino	1 wk	Fri 9/15/23	Thu 9/21/23	2
4		Completing Requirment to Follow Black Line	1 wk	Fri 9/22/23	Thu 9/28/23	3
5		Performing Maintenance on Vehicle	1 wk	Fri 9/29/23	Thu 10/5/23	4
6		Discussing Obstacle Detection with Professor Sunda	1 wk	Fri 10/6/23	Thu 10/12/23	5
7		Completing Obstacle Avoidance, Rectifying Detection	1 wk	Fri 10/13/23	Thu 10/19/23	6
8		Completing Obstacle Detection	1 wk	Fri 10/20/23	Thu 10/26/23	7
9		Testing Functional Requirments	1 wk	Fri 10/27/23	Thu 11/2/23	8
10		Condensing/Commenting Vehicle Code/Meeting with Advisor	1 wk	Fri 11/3/23	Thu 11/9/23	9
11		Creating PowerPoint fo Vehicle/Meeting with Professor Grogan	1 wk	Fri 11/10/23	Thu 11/16/23	10
12		Adding Content to Research Paper	2 wks	Fri 11/17/23	Thu 11/30/23	11
13		Presentation Day	1 day	Fri 12/1/23	Fri 12/1/23	12
14		Final Report	1 day	Wed 12/13/23	Wed 12/13/23	13

Table 1 shows the Gantt chart of our Autonomous Delivery Vehicle